

石器时代战斗系统与宠物养成机制的源码还原与定量分析

Rode (QQ:81881788)

2026 年 4 月 15 日

摘要

本文基于 `gmsv` 源码，研究《石器时代》中战斗系统与宠物养成机制的定量结构。全文首先建立统一的研究对象、术语、符号与证据口径，随后分别分析宠物成长、转生融合、行动顺序、物理概率、法术命中、骑宠伤害、装备字段以及动态属性状态。研究表明，多个长期流传的玩家经验口径只有在特定变量位置与编译路径下才成立，而宠物培养与蛋喂药问题在本质上属于受多次取整、总和上限与比例回分配共同约束的离散优化问题。本文尝试在源码证据与定量分析之间建立一条统一的研究链条，以便将经验性判断还原为可验证的机制命题。

关键词：石器时代；源码分析；战斗系统；宠物养成；离散优化

目录

第一章 绪论	1
1.1 研究背景	1
1.2 研究问题	1
1.3 研究方法与证据标准	1
1.4 主要贡献	2
1.5 全文结构	2
第二章 研究对象、证据口径与方法框架	3
2.1 研究对象与版本边界	3
2.2 符号体系与术语映射	3
2.3 离散运算的统一表示	5
2.4 方法框架	5
第三章 宠物基础成长与显示属性映射	6
3.1 成长档与显示属性映射的基本结构	6
3.2 初始化随机与 Rank 门槛	7
3.3 成长约束的两种来源	9
3.4 蛋基础成长与孵化后成长的区分	10
第四章 宠物养成链：转生、融合与蛋喂养	11
4.1 转生作为成长档重构	11
4.2 融合作为逐步截断变换	13
4.3 喂药与孵化的两级压缩结构	15
4.4 孵化结果的解释口径	17
4.5 养成流水线的统一表示	17
第五章 宠物养成到战斗能力的映射	18
5.1 从 (V, S, T, D) 到战斗属性的映射	18
5.2 从转生、融合到战斗能力的长期收益	18
5.3 从蛋喂养到战斗目标的定向优化	19
5.4 宠物忠诚失控 (nono) 与动作改写	19
5.5 不同培养目标下的宠物类型	21
5.6 人物与宠物协同	21

第六章 离散优化模型、案例与策略含义	23
6.1 优化问题的形式化	23
6.2 蛋喂养最优化	23
6.3 战斗目标反推	24
6.4 转生与融合策略比较	24
6.5 案例研究	25
第七章 战斗有效属性与状态变量	26
7.1 有效属性的三层口径	26
7.2 单位身份与公式切换	26
7.3 关键状态变量及其作用入口	27
7.4 状态槽的存储方式与回合更新	27
第八章 行动顺序与速度分支结构	29
8.1 参照敏捷的来源与骑宠权重	29
8.2 动作分支与顺序公式	29
8.3 职业技能的速度口径	31
8.4 先手区间与阈值判断	32
第九章 物理命中、回避、会心与反击	34
9.1 普通回避链的判定分层	34
9.2 回避公式与对象分支	35
9.3 会心率、武器字段与真实概率	41
9.4 反击的触发条件与离散公式	43
9.5 连续化分析与主导趋势	44
9.6 离散数值图示与结果对照	45
第十章 法术、精灵与职业技能的命中和伤害分层	47
10.1 普通法术命中的对象分支	47
10.2 精灵与普通法术的同构命中链	48
10.3 职业法术命中与熟练度入口	48
10.4 二段命中与状态技能的后续写入	49
10.5 命中层与伤害层的分离	50
第十一章 骑宠混合与物理伤害结构	51
11.1 骑宠攻防混合的基本结果	51
11.2 骑宠敏捷混合与攻守差异	52
11.3 基础物伤的三段结构	52
11.4 防御动作与后置减伤链	53
11.5 属性层、场地与结界减伤	54
11.6 会心事件的伤害层附加	59
11.7 变量收益的结构性差异	59

第十二章 装备字段、作用机制与案例研究	61
12.1 装备字段的分类口径	62
12.2 直接属性字段的工作值入口	63
12.3 传统异常抗性的适用边界	63
12.4 行动顺序修正字段的独立入口	64
12.5 会心字段与武器槽量纲	64
12.6 反击字段与“格挡率”文本偏差	65
12.7 伤 +X / 伤-X 字段的尾段随机修正	65
12.8 ITEM_MODIFYAVOID 的保守解释	66
12.9 本章小结	66
第十三章 属性系统的动态状态：场地、结界、加强与反转	67
13.1 动态属性系统的整体框架	67
13.2 场地作为战场级状态	67
13.3 四属性结界的覆盖写入与减伤读取	68
13.4 属性加强的空实现结论	70
13.5 属性反转的标志位与快照分离	70
13.6 多次状态命中的属性演化	71
13.7 动态属性系统的状态转移模型	71
第十四章 综合讨论	73
14.1 经验口径与源码口径之间的张力	73
14.2 离散机制分析的理论含义	73
14.3 适用边界与后续拓展方向	74
第十五章 结论	75
15.1 主要研究结论	75
15.2 对经验口径的修正	75
15.3 研究局限与后续工作	76
附录 A 符号表	77
附录 B 公式总表	80
附录 C 源码索引	83

第一章 绪论

1.1 研究背景

《石器时代》作为长期运营的网络游戏，其战斗系统与宠物养成系统在玩家社群中积累了大量经验性判断。这些判断一方面来源于长期实战观察，另一方面又混杂了旧版本记忆、私服转述以及不同编译分支之间的口径迁移。由此形成的机制理解往往具有三个共同特征：只给出经验结论而不说明变量进入公式的顺序；将局部成立的结论外推到所有对象组合与动作分支；忽略取整、上限、状态覆盖与回合重算所造成的离散边界。

上述问题在战斗系统与宠物养成系统中尤为突出。行动顺序究竟读取人物敏捷还是骑宠调整后的参照敏捷，普通物品、精灵、普通法术与职业技能是否适用同一速度分支，会心、反击、传统异常抗性、职业控制抗性与装备字段之间如何对应，以及宠物转生、融合、蛋喂药与孵化流程中的各类取整与压缩何时发生、如何改变最优方案，这些都是长期存在而又难以仅凭经验统一回答的问题。

1.2 研究问题

本文围绕两个彼此关联的研究问题展开。其一，在当前 `gmsv` 源码口径下，《石器时代》的战斗与养成机制能否被统一还原为一组清晰的变量、状态与分段函数系统。其二，在这一统一框架之内，玩家社群中长期流传的经验说法有哪些能够被源码直接支持，哪些只在特定条件下成立，哪些则需要改写为更保守的定量命题。

与一般的经验整理不同，本文并不以“汇总结论”为最终目标，而是关注变量进入何处、按何种顺序生效，以及在何处被取整、裁切或覆盖。正是在这些位置上，源码结构决定了经验口径成立的条件与边界。

1.3 研究方法与证据标准

本文以当前持有的 `gmsv` 源码为最高优先级证据，并以项目 `sa_pet_assistant` 中已与源码对齐的实现作为辅助验证环境。研究方法包括四个相互衔接的层面：其一，逐段核对关键函数、宏分支与状态槽位，完成源码还原；其二，将分散在不同文件中的变量映射到统一的术语与符号体系；其三，将行动顺序、命中、会心、反击、伤害与喂药压缩等

机制写成可分析的分段表达；其四，在此基础上进一步讨论状态转移、数值边界与离散优化问题。

在结论表述上，全文采取保守口径：对于当前源码中尚未直接实现、尚未完全核清，或可能因不同编译路径而改变的机制，仅给出保守结论，而不以经验材料补写公式。

1.4 主要贡献

本文的贡献可概括为四点。第一，建立统一的研究对象、术语、符号与证据口径，使玩家术语、工具实现与源码变量之间能够形成稳定对应。第二，系统还原行动顺序、物理概率、法术命中、骑宠混合、装备字段与动态属性状态等关键战斗机制。第三，将宠物成长、转生、融合、蛋喂药与孵化纳入同一离散分析链条，说明其如何映射到高等级阶段的战斗能力。第四，在机制还原的基础上，将若干典型玩家问题转化为可计算的离散优化问题，从而把经验性判断提升为可验证的定量命题。

1.5 全文结构

全文结构如下。第 2 章建立研究对象、证据口径与方法框架。第 3 章至第 6 章集中讨论宠物侧问题，依次分析宠物基础成长、转生融合与蛋喂药、养成到战斗能力的映射，以及离散优化模型与案例。第 7 章作为战斗部分的变量准备，整理战斗有效属性与状态变量。第 8 章至第 13 章依次分析行动顺序、物理概率、法术命中分层、骑宠混合后的伤害结构、装备字段以及动态属性系统。第 14 章给出综合讨论，第 15 章总结全文并说明局限与后续工作。

全文的论证顺序遵循“统一口径、还原机制、讨论边界、再做综合分析”的路径。只有在澄清变量入口、离散规则与状态刷新顺序之后，关于最优方案、装备收益与战斗构筑的结论才具有可验证性。

第二章 研究对象、证据口径与方法框架

本章建立全文的研究对象、符号体系与证据口径，并说明随机、取整与上下限在分析中的统一表示方式。后文关于战斗机制与养成模型的推导，均以本章给出的对象边界和术语规则为前提。

2.1 研究对象与版本边界

本文研究对象包括五类：人物、宠物、骑宠状态下的人物-宠物组合、融合蛋，以及战斗中的敌方单位。这些对象的机制彼此关联：宠物成长档位影响战斗敏捷，骑宠状态混合人物与宠物的攻防敏属性，蛋喂药和孵化则通过成长率映射到命中、回避、会心和伤害。

本文的主要源码依据是当前项目持有的 `gmsv` 代码树及其配套文档。对于已验证的机制，正文直接给出公式或时序结论；对于尚未完全核实的机制，正文明确采用保守表述。在骑宠、职业技能与属性状态系统相关部分，编译开关会改变具体实现，因此全文默认采用当前项目已核对的主要口径：骑宠三围与战斗伤害以 `_BATTLE_NEWPOWER` 分支为主，必要时指出 `_BACK_VERSION` 的参与情况。

由此可见，本文构建的是一个在当前源码与当前编译路径下自洽、可计算且可验证的数值模型，而非对“所有可能服务器实现”的并行还原。若无特别说明，正文中“成立”“等于”“上限”“下限”等表述，均指“在当前 `gmsv` 源码与编译口径下成立”。

2.2 符号体系与术语映射

本文采用表 2.1 和表 2.2 所示的统一符号体系。前者列出成长与对象记号，后者列出战斗概率记号；完整符号索引、别名与源码变量对应关系保留在附录中。

在记号选择上，宠物基础成长档统一记作 g ，蛋基础成长统一记作 b ，孵化后成长统一记作 h 。玩家常用的 (V, S, T, D) 在正文中作为玩家口径别名出现，但不与 g 并列为主记号。

表 2.1: 全局核心符号与成长相关记号

记号	含义	说明
V, S, T, D	宠物四项成长率	分别对应生命、攻击、防御、敏捷方向的成长来源
g	基础成长档	正文主记号；玩家口径常把它写成 (V, S, T, D)
$VSTD$	四项成长和	定义为 $V + S + T + D$
L_o	宠物等级	用于区分蛋、孵化后宠物、转生前宠物等阶段
b	蛋基础成长	融合完成后、喂药前或喂药后的蛋面值
h	孵化后成长	不含出生后额外随机加点的孵化结果
HP, ATK, DEF, SPD	宠物面板四围	分别表示界面中显示的生命、攻击、防御、敏捷数值
x_1, x_2, x_3, x_4	线性目标函数权重	用于喂药优化、养成反推等问题

表 2.2: 战斗核心符号与概率记号

记号	含义	说明
att_{dex}	攻击方固定敏	在玩家 PVP 简化模型中通常对应 WORKFIXDEX
def_{dex}	防守方固定敏	在玩家 PVP 简化模型中通常对应防守方 WORKFIXDEX
L	幸运修正	依上下文代表攻方或守方幸运，玩家有效范围通常为 $1 \sim 5$
C	武器会心值	源码中的武器会心附加参数
F	反击武器系数	玩家反击率中的武器对武器系数
α, β, γ	概率参数	分别对应回避、会心、反击公式中的主参数
p_{duck}	普通回避率	不含独立前判的普通闪避概率
p_{hit}	普通命中率	通常定义为 $100\% - p_{duck}$
p_{crit}	会心率	当前为普通物理会心判定
$p_{counter}$	反击率	在满足反击前提后的成功概率

在战斗层面，本文区分两类敏捷变量：WORKFIXDEX 用于命中、回避、会心、反击等概率判定；WORKQUICK 用于行动顺序计算。通常 WORKQUICK 由 WORKFIXDEX 经战斗修正后生成，但二者并不总是相等。后文讨论命中、回避或会心时使用“固定敏”，讨论乱敏与先手时使用“战斗速度敏”。

术语映射则采用三层并行规则：源码层保持 WORKFIXDEX、WORKQUICK、BATTLE_COM_GUARD 等标识符的字面一致性；数学层使用 att_{dex} 、 def_{dex} 、 p_{duck} 、 g 等记号进行推导；语

义层使用“固定敏”“战斗速度敏”“普通回避率”“咒术动作分支”等中文术语解释机制含义。三者之间保持稳定对应，是后文避免概念滑移的基础。

2.3 离散运算的统一表示

战斗与宠物系统中的争议往往源于三类离散操作的位置：随机、截断和上限裁切。源码中的随机宏输出恒为整数，因此任何随机值在进入后续计算前都已经回到整数域。与此同时，源码中大量百分比修正虽然在中间以浮点形式出现，但只要最终回写到整型变量，就会表现为向下截断而非四舍五入。例如，

$$att'_{dex} = \lfloor 0.8 att_{dex} \rfloor, \quad def'_{dex} = \lfloor 0.6 def_{dex} \rfloor. \quad (2.1)$$

为统一书写上限与下限，本文使用裁切算子

$$\text{clip}_{[a,b]}(z) = \min\{\max\{z, a\}, b\}. \quad (2.2)$$

这一写法可以将“基础体”与“边界约束”分离。例如回避率、会心率、成长压缩和蛋喂药分配中的总和限制，都可以在保留主体结构的同时，用统一的裁切记号表达其边界效应。

需要特别指出的是，离散结构最常出现于三个位置：身份修正所带来的单边缩放与截断；成长与喂药过程中单项上限和总和上限造成的压缩；概率链条中先取整后裁切的上限效应。平台、跳变和局部非单调现象，正是在这些位置上被引入系统之中。

2.4 方法框架

本文的方法框架由四个相互衔接的步骤构成。第一步是源码还原，即逐段核对关键函数、宏分支、状态槽位和写入时序。第二步是符号化整理，即将分散在不同文件中的变量、状态与动作统一到同一记号体系之下。第三步是机制表达，即将行动顺序、命中、会心、反击、伤害以及成长压缩等过程写成可分析的分段表达和状态转移结构。第四步是综合解释，即在上述结果基础上讨论局部非单调、边界条件、样例装备与离散优化问题。

在这一框架中，源码始终是最高优先级证据，辅助验证环境只用于复核已经由源码支持的结论。也正因为如此，后文所有高层概括都必须能够回指到具体的公式、状态写入点或读取链，而不能脱离源码独立成立。

第三章 宠物基础成长与显示属性映射

本章从源码角度重建宠物基础成长进入可见面板的映射过程。分析重点不在于罗列单项成长率，而在于说明成长档、随机初始化、等级推进与显示公式之间的层次关系。只有先澄清这一基础映射，后文关于转生、融合、喂药与战斗表现的讨论才具有统一的变量口径。

3.1 成长档与显示属性映射的基本结构

在本文中，宠物的基础成长方向由四维向量

$$\mathbf{g} = (V, S, T, D) \quad (3.1)$$

表示，其中四个分量分别对应生命、攻击、防御与敏捷方向的成长率。对应的总和记为

$$VSTD = V + S + T + D. \quad (3.2)$$

$VSTD$ 虽然能够概括成长总量，但并不足以唯一表征宠物。相同总和对应的四维分配不同，成长方向与实战能力即可出现明显差异。因此，本文在讨论中同时保留总和视角与分量视角。

宠物原始成长点数到显示属性的换算为

$$HP = \left\lfloor \frac{4V_r + S_r + T_r + D_r}{100} \right\rfloor, \quad (3.3)$$

$$ATK = \left\lfloor \frac{S_r + 0.1T_r + 0.1V_r + 0.05D_r}{100} \right\rfloor, \quad (3.4)$$

$$DEF = \left\lfloor \frac{T_r + 0.1S_r + 0.1V_r + 0.05D_r}{100} \right\rfloor, \quad (3.5)$$

$$SPD = \left\lfloor \frac{D_r}{100} \right\rfloor, \quad (3.6)$$

其中 (V_r, S_r, T_r, D_r) 为原始累计点数。式 (3.6) 表明四项成长并非彼此独立地映射到四项面板属性： V 主要进入生命，同时少量进入攻击与防御； D 主要进入敏捷，同时少量进入攻击与防御。该映射是带交叉项的线性变换叠加整数截断，因此具有两项性质：(1) 多对一，即不同原始点数组合在向下取整后可产生相同显示属性；(2) 层级性，即面板属性由随机过程累积得到的 (V_r, S_r, T_r, D_r) 决定，而非由基础成长率 (V, S, T, D) 直接决定。由显示属性反推成长率时，只能得到候选集合而不能唯一还原。

3.2 初始化随机与 Rank 门槛

3.2.1 一级初始化的离散来源

普通宠物的一级初始化包含三步。第一步是四维独立的 ± 2 波动：

$$\varepsilon_i \in \{-2, -1, 0, 1, 2\}, \quad g_i^{(s)} = g_i + \varepsilon_i, \quad (3.7)$$

其中 $g_i^{(s)}$ 表示真正存入成长档的“生效成长率”。第二步依据波动后的四维总和判定 Rank，而不是依据基础输入的 $VSTD$ 判定。第三步是在出生时进行一次总计 10 点的随机分配，每次随机给四维中的某一项加 1。当前项目配套的宠物成长模拟，为与既有模拟口径保持一致，在此额外采用了“单项最多额外得到 4 点”的限制；需要强调的是，这一限制属于工具侧模拟设定，并非本文已直接从服务器源码验证出的硬编码上限。若按“10 次独立、每次以 1/4 概率落到某一维”的无上限模型考察，对预先指定的一项有

$$X \sim \text{Binomial}(10, 1/4), \quad \Pr(X \geq 5) = \sum_{k=5}^{10} \binom{10}{k} (1/4)^k (3/4)^{10-k} \approx 7.81\%. \quad (3.8)$$

进一步地， $\Pr(X \geq 6) \approx 1.98\%$ ， $\Pr(X \geq 7) \approx 0.35\%$ 。因此，在无上限随机模型下，“单项超过 4 点”并非不可能事件，只是对预先指定的某一维而言概率较低。若将出生时随机加点记作 e_i ，初始化参数记作 $INITNUM$ ，则一级原始点数满足

$$g_{r,i}^{(1)} = INITNUM \cdot (g_i^{(s)} + e_i). \quad (3.9)$$

这说明一级面板已经混合了基础成长率、 ± 2 波动、出生随机加点与初始化尺度四类信息，出生时刻的显示属性本身就不是单一成长档的直接呈现。

3.2.2 普通宠与转生宠的 Rank 门槛差异

Rank 是宠物成长系统中的关键中介变量。普通宠物（0 转宠）与转生宠使用不同的 Rank 判定门槛表。

普通宠物的 Rank 由波动后的 $VSTD$ 判定，分界如表 3.1 所示。

表 3.1: 普通宠物（0 转宠）的 Rank 判定表

波动后 $VSTD$	Rank
≥ 100	0
≥ 95	1
≥ 90	2
≥ 85	3
≥ 80	4
< 80	5

对于转生宠，Rank 判定门槛会发生变化，其分界如表 3.2 所示。

表 3.2: 转生宠的 Rank 判定表

波动后 $VSTD$	Rank
≥ 130	0
≥ 100	1
≥ 95	2
≥ 85	3
≥ 80	4
< 80	5

两表的关键差异在于 Rank 0 门槛由 100 提高到 130，从而改变了高成长转生宠的判定标准。Rank 判定对象始终是波动后的生效成长总和，但门槛表会随宠物是否已经转生而变化。首转计算原宠 Rank 时仍使用普通表；转生后的新成长率重新进入成长流程时，则按转生宠 Rank 表判定（详见第 4 章）。

Rank 一经确定，就会进入升级随机的倍率表。当前实现中，不同 Rank 对应的随机倍率区间如表 3.3 所示。

表 3.3: 不同 Rank 的升级倍率范围

Rank	$fRand$ 对应整数区间
0	$[450, 500] \times 0.01$
1	$[470, 520] \times 0.01$
2	$[490, 540] \times 0.01$
3	$[510, 560] \times 0.01$
4	$[530, 580] \times 0.01$
5	$[550, 600] \times 0.01$

因此，Rank 并非单纯的标签变量，而是后续升级随机参数的入口。同一只基础宠物若因 ± 2 波动跨过 Rank 边界，其后续升级路径将整体改变。

3.2.3 两类成长流程的共同部分与差异

0 转宠与转生宠真正不同的是 Rank 判定门槛，而非整个成长引擎。一级初始化公式、出生 10 点随机、升级时的 10 点随机、 $fRand$ 倍率表与显示公式均相同，变化的仅是 Rank 表。两者的完整对照见表 3.4。

表 3.4: 0 转宠与转生宠的成长口径对照

比较项目	0 转宠	转生宠
适用对象	普通捕捉宠、普通孵化宠、尚未进入转生路径的宠物	已完成转生并重新进入成长流程的宠物
Rank 判定对象	波动后的生效成长总和 $\sum_i g_i^{(s)}$	波动后的生效成长总和 $\sum_i g_i^{(s)}$
Rank 门槛	使用表 3.1: Rank 0 门槛为 100	使用表 3.2: Rank 0 门槛提高到 130
一级出生 10 点随机	总计 10 点; 当前工具模拟默认额外采用单项最多 4 点的限制	同左
升级时 10 点随机	总计 10 点, 不设单项 4 点限制	同左
$fRand$ 区间表	使用表 3.3	与 0 转宠相同, 仍使用表 3.3
显示属性换算	使用式 (3.6)	与 0 转宠相同, 仍使用式 (3.6)

此外还需区分 Rank 的两个使用场景: (1) 转生公式内部计算 F_x 时对原宠判 Rank, 其中首转使用普通表、二转使用转生表; (2) 转生完成后, 新成长率 $g^{(new)}$ 重新进入成长流程时统一按转生表判 Rank。二者作用阶段不同, 不宜混为一谈。

3.2.4 升级阶段的随机增量

每次升级时系统再执行两步: 一次总和为 10 的随机加点 (不设单项限制), 以及从 Rank 对应区间中抽取倍率 $fRand$ 。若第 ℓ 级到第 $\ell + 1$ 级的随机加点记为 $p_i^{(\ell)}$, 则升级增量为

$$\Delta g_{r,i}^{(\ell)} = \left\lfloor (g_i^{(s)} + p_i^{(\ell)}) fRand^{(\ell)} \right\rfloor. \quad (3.10)$$

由式 (3.10) 可知, 高等级宠物的最终属性对应的是一个分布而非固定点; 成长模拟中输出分位数的做法, 正是对此一分布结构的数值呈现。

3.3 成长约束的两种来源

宠物系统中的上限可分为两类: 随机模型侧约束, 例如出生 10 点随机分配中“单项最多 4 点”的工具侧限制; 以及成长档压缩约束, 例如蛋喂药、孵化中的单项上限与总和上限。前者只影响初始点的离散分布, 后者则直接改变成长档本身并贯穿整个生命周期。因此, 养成优化的自然目标应是孵化后成长 h , 而非蛋基础成长或出生后的一级面板。

在这类带上限的整数系统中, 单项上限、总和上限与取整顺序都会在可行域上制造折角与跳点, 因而后续问题应理解为离散约束优化, 而非连续配比问题。

3.4 蛋基础成长与孵化后成长的区分

融合与喂药系统引入蛋基础成长与孵化后成长两个概念。蛋基础成长记为

$$\mathbf{b} = (b_V, b_S, b_T, b_D), \quad (3.11)$$

它表示融合完成后、喂药前或喂药后的蛋面数值；再记孵化后成长为

$$\mathbf{h} = (h_V, h_S, h_T, h_D), \quad (3.12)$$

它表示按孵化公式折算之后、但尚未进入出生后 10 点随机加点之前的成长档。当前工具中的优化问题和大多数比较均以 $\mathbf{h}$ 为对象，而不直接以一级显示属性为对象。

无喂药时，孵化是以 $\mathbf{b}$ 为输入、以 $\mathbf{h}$ 为输出的整数映射；有喂药时，还需经过药效折算、单项裁切、总和压缩和按比例回分配（详见下一章）。孵化后成长 $\mathbf{h}$ 是进入成长流程的成长档；出生后状态还将叠加 ± 2 波动、Rank 判定和出生随机加点。

综上，蛋面成长只是中间状态，经孵化映射后生成的 $\mathbf{h}$ 才是真正进入后续成长流程的成长档。本章所建立的“两层结构”即成长档本身，以及由成长档驱动的随机成长过程，将构成后续章节的共同基础。

第四章 宠物养成链：转生、融合与蛋喂养

转生、融合与蛋喂药在源码中分散为若干彼此衔接的模块，但从数值结构上看，它们共同构成了一条连续的宠物养成链。原宠可以通过转生重构成长档，也可以通过融合生成蛋基础成长；蛋基础成长再经喂药与孵化映射为新的成长档，随后重新进入普通成长流程。本章据此将相关机制统一表述为一组可组合的离散变换，为后文分析高等级面板与战斗能力的形成路径奠定基础。

4.1 转生作为成长档重构

4.1.1 输入变量与整体流程

转生模块的输入包括原宠成长率 $\mathbf{g} = (g_1, g_2, g_3, g_4)$ 、MM 向量 $\mathbf{m} = (m_1, m_2, m_3, m_4)$ 、宠物等级 L_v 、转生次数以及转生能力上限 $A_{\max}$ （首转默认 150，二转默认 200，可人工覆盖）。

转生首先生成一组新的基础成长率，该成长率再经历普通出生与升级流程。记转生后的确定性成长率为 $\mathbf{g}^{(new)}$ ，则逻辑链可写为

$$(\mathbf{g}, \mathbf{m}, L_v, \text{转生次数}, A_{\max}) \longrightarrow A \longrightarrow \mathbf{g}^{(new)} \longrightarrow \text{普通成长流程}. \quad (4.1)$$

其中 A 即当前实现中的转生能力值，对应项目内部变量 `ans`。因此，转生并不是对最终属性的直接修补，而是在成长档层面先完成一次离散重构，再经普通成长流程传导到高等级数值。

4.1.2 Rank、Fx 与转生能力值的形成

设原宠总和为

$$T_2 = \sum_{i=1}^4 g_i, \quad (4.2)$$

MM 总和经上限裁切后为

$$T_1 = \min \left\{ \sum_{i=1}^4 m_i, 150 \right\}. \quad (4.3)$$

首转时，原宠 Rank 按普通宠物的 Rank 表判定；二转时，则改按转生宠 Rank 表判定。记此 Rank 为 r ，则项目实现中的

$$F_x = \lfloor (5 - r) \times 1.2 \rfloor + 5. \quad (4.4)$$

将等级截到 130 以内，记

$$\tilde{L} = \min\{L_v, 130\}, \quad (4.5)$$

则转生能力值可整理为

$$A = \left\lfloor 1.3 \left(\frac{T_1}{100} \right)^5 \right\rfloor + T_2 + \left\lfloor \frac{\tilde{L} - 100}{F_x} \right\rfloor. \quad (4.6)$$

再施加转生能力上限，得到

$$A^* = \min\{A, A_{\max}\}. \quad (4.7)$$

式 (4.6)–(4.7) 表明，MM 总和通过五次方项提供高阶放大，而不是线性叠加；等级补正只在 100 级以上生效，并在 130 级后饱和；最终结果还受到 $A_{\max}$ 的上限约束。

转生能力值参数扫描表（原宠总和=120，等级=130）

MM 总和	首转原始 A	首转封顶后	二转原始 A	二转封顶后
80	122	122	123	123
100	123	123	124	124
120	125	125	126	126
130	126	126	127	127
140	128	128	129	129
150	131	131	132	132

原宠固定为 (37, 33, 28, 22)；MM 维持 4:3:2:1 分配比例。

图 4.1: 转生能力值参数扫描表。固定原宠成长总和为 120、宠物等级为 130，并令 MM 的二维分配保持 4:3:2:1 比例。表中同时列出首转与二转的原始转生能力值和上限裁切结果，用于展示 Rank 表差异与能力上限的共同影响。

由于四维结果各自向下取整，分配后的四项总和一般小于 A^* ，产生 1 至 3 点的取整损失。图 4.2 的扫描显示，四维总和随 A^* 以台阶方式增长。

转生能力值分配后的取整损失表

A*	分配后四维	四维总和	取整损失
140	46/39/31/22	138	2
145	47/40/32/23	142	3
150	49/42/33/24	148	2
155	51/43/34/25	153	2
160	52/44/36/26	158	2

原宠固定为 (37, 33, 28, 22)；MM 总和固定 150，分配比例 4:3:2:1。

图 4.2: 转生能力值分配后的取整损失表。固定原宠成长为 (37, 33, 28, 22)、MM 总和为 150 且分配比例为 4:3:2:1，展示不同 A^* 在按权重分配并逐项向下取整后，四项总和与原能力值之间的差额。

4.1.3 能力值向四维成长的分配

A^* 按“原宠四倍权重、MM 一倍权重”分配到四维。记总权重为

$$W = T_1 + 4T_2, \quad (4.8)$$

则每一维的新成长率为

$$g_i^{(new)} = \left\lfloor \frac{A^*(m_i + 4g_i)}{W} \right\rfloor, \quad i = 1, 2, 3, 4. \quad (4.9)$$

由于权重中原宠占四倍、MM 仅占一倍，MM 的作用更接近于对原结构的方向性修正，而非对原宠结构的完全替换。同时， $\mathbf{g}^{(new)}$ 仍将经历 ± 2 波动、Rank 判定、出生随机加点与升级随机，因此转生后的高等级属性依然应理解为分布而非单一点值。

4.2 融合作为逐步截断变换

4.2.1 主宠、副宠与低等级折扣次序

融合模块的输入为主宠与一至两只副宠的等级及成长率。等级低于 80 的参与宠需先逐项做低等级折扣：

$$g'_i = \lfloor 0.8g_i \rfloor. \quad (4.10)$$

关键在于低等级折扣先于后续贡献系数执行：每只低等级宠物先单独完成截断，再进入主宠贡献或副宠平均步骤。由此可见，融合并非简单的线性平均，而是由多层取整算子串联而成的离散变换。

4.2.2 主宠贡献、副宠贡献与蛋基础成长

设主宠折扣后的成长向量为 $\mathbf{g}^{(M)}$ ，副宠折扣后的成长向量分别为 $\mathbf{g}^{(S1)}$ 、 $\mathbf{g}^{(S2)}$ 。若副宠数量为 $n_s \in \{1, 2\}$ ，则先求逐项和

$$s_i = \sum_{k=1}^{n_s} g_i^{(Sk)}, \quad (4.11)$$

再逐项做整数平均

$$\bar{s}_i = \left\lfloor \frac{s_i}{n_s} \right\rfloor. \quad (4.12)$$

随后，主宠贡献与副宠贡献分别为

$$c_i^{(M)} = \left\lfloor 0.6g_i^{(M)} \right\rfloor, \quad (4.13)$$

$$c_i^{(S)} = \left\lfloor 0.4\bar{s}_i \right\rfloor. \quad (4.14)$$

最终蛋基础成长为

$$b_i = c_i^{(M)} + c_i^{(S)}. \quad (4.15)$$

式 (4.12) 与式 (4.14) 的先后次序至关重要。“先平均再乘 0.4”与“先乘 0.4 再平均”在整数系统中并不等价。当前实现采用前者，因此融合模型应理解为次序不可交换的逐步取整算子。

低等级副宠融合时的取整顺序对比表

案例	副宠等级	gmsv 顺序	先平均再 0.8 (误解)
案例A	60/70	33/29/24/20	33/29/24/20
案例B	79/79	34/28/24/20	34/28/24/20
案例C	50/78	31/28/22/17	31/28/22/17

各行给出蛋基础 V/S/T/D。gmsv 会先对每只低等级副宠做折扣，再参与平均。

图 4.3: 低等级副宠参与融合时的取整顺序对比。表中给出三组副宠案例，并比较 gmsv 实际采用的“各自先做 0.8 折扣，再平均”与常见误解“先平均，再统一做 0.8 折扣”所得到的蛋基础成长。可以看到，仅仅交换取整顺序就足以改变最终蛋面。

4.3 喂药与孵化的两级压缩结构

4.3.1 药效累加与工作值折算

蛋喂药的输入为蛋基础成长 $\mathbf{b}$ 与 4×5 喂药方案矩阵。记第 i 维在第 ℓ 级药上的颗数为 $c_{i\ell}$ ，第 ℓ 级药的原始值为 e_ℓ ，则药效原始和为

$$r_i = \sum_{\ell=1}^5 c_{i\ell} e_\ell. \quad (4.16)$$

默认 e_ℓ 依次为 25, 50, 75, 100, 125，总颗数固定为 40。药效经折算后为

$$u_i = \min \left\{ \left\lfloor \frac{7r_i}{1000} \right\rfloor, 60 \right\}. \quad (4.17)$$

折算单项上限为 60。不同等级药在原始值层面虽然可以线性相加，但在折算与单项上限进入后，最终结果会出现显著的离散差异。

4.3.2 工作值压缩与比例回分配

记折算后的总和为

$$U = \sum_{i=1}^4 u_i. \quad (4.18)$$

若 $U \leq 50$ ，则直接令 $w_i = u_i$ ；若 $U > 50$ ，则先做整体压缩：

$$w_i = \left\lfloor \frac{50u_i}{U} \right\rfloor. \quad (4.19)$$

w_i 再经“半折算 + 按比例回分配”生成孵化后成长。记

$$T_1 = \sum_{i=1}^4 w_i, \quad T_2 = \sum_{i=1}^4 b_i, \quad (4.20)$$

并定义

$$T = \left\lfloor \frac{T_1}{2} \right\rfloor + T_2. \quad (4.21)$$

当前实现对每一维使用

$$f_i = b_i + \left\lfloor \frac{w_i}{2} \right\rfloor, \quad (4.22)$$

再计算增量

$$\Delta_i = \left\lfloor \frac{f_i}{T} T_1 \right\rfloor, \quad (4.23)$$

从而得到初步孵化成长

$$h'_i = \text{clip}_{[1,60]}(b_i + \Delta_i). \quad (4.24)$$

式 (4.24) 中 $w_i//2$ 和 $T_1//2$ 均为整数除法，因此药效并入蛋成长时会产生额外的整数损失。

4.3.3 孵化总和压缩与最终成长档

记初步孵化成长总和为

$$H' = \sum_{i=1}^4 h'_i. \quad (4.25)$$

若 $H' \leq 150$ ，则最终孵化后成长直接取 $h_i = h'_i$ ；若 $H' > 150$ ，则再施加一次总和压缩：

$$h_i = \text{clip}_{[1,60]} \left(\left\lfloor \frac{150h'_i}{H'} \right\rfloor \right). \quad (4.26)$$

孵化后成长由式 (4.17)–(4.26) 共同决定，本质上是一个同时受到单项上限、总和上限、比例回分配和多次向下取整约束的组合优化问题。

在默认参数下，所有药的原始总和最多为 $40 \times 125 = 5000$ ，折算后四维工作值总和满足

$$\sum_i u_i \leq \left\lfloor \frac{7 \times 5000}{1000} \right\rfloor = 35, \quad (4.27)$$

因此，式 (4.19) 所表示的 50 压缩在默认规则下并不触发；真正起决定作用的总和约束，是最终孵化阶段的 150 总和压缩。

默认 40 药规则下的孵化案例表

案例	u_i 总和	触发 50 压缩	最终总和	触发 150 压缩	最终 h
均衡蛋面	4	否	120	否	30/30/30/30
高基础	32	否	150	是	45/40/35/30
压缩压力	32	否	148	是	53/53/21/21

默认药效参数下，折算工作值总和最多只有 35，因此 50 压缩在实证上处于休眠状态。

图 4.4: 默认 ‘40’ 药规则下的孵化案例表。表中列出不同蛋基础成长和喂药方案下的折算工作值总和、‘50’ 压缩是否触发、最终孵化总和以及 ‘150’ 压缩是否触发。它直接展示了一个实证事实：在默认药效参数下，‘50’ 压缩不触发，而真正决定结果的常是最终 ‘150’ 总和压缩。

若进一步把问题聚焦到 “单项最终能否到 60”，则还会出现另一层阈值现象：即使某一维的喂药全部集中，只要其余三项总和过高，最终 ‘150’ 压缩仍会把该维重新压回去。图 4.5 给出一个固定实验口径：把 ‘40’ 颗五级药全部喂给敏捷维，并让其余三项基础成长尽量均分。表中列出的最小基础值阈值，正可以用来说明 “为什么蛋面不到某个点，孵化后单项就上不去 60”。

默认 40 药规则下单项到 60 的基础阈值表

其余三项总和	D 最小基础值	最终孵化四维	最终总和
30	38	14/14/14/60	102
45	41	20/20/20/60	120
60	43	25/25/25/60	135
75	54	30/30/30/60	150
90	-	-	-

固定把 40 颗五级药全部喂给 D，其余三项基础成长尽量均分。

图 4.5: 默认 ‘40’ 药规则下单项到达 60 的基础阈值表。固定把 ‘40’ 颗五级药全部投入敏捷维，并令其余三项基础成长尽量均分，比较在不同 “其余三项总和” 下，敏捷维最终到达 60 所需的最小蛋面基础值。

4.4 孵化结果的解释口径

孵化过程可分为两层：确定性孵化成长 h ，以及进入出生流程后再叠加 ± 2 波动、Rank 判定与 10 点随机加点的随机出生状态。本文将前者作为优化对象，由此将可控的喂药方案比较与出生阶段的随机实现明确区分开来。

4.5 养成流水线的统一表示

综合前述各节，本文将宠物养成系统写成如下流水线：

$$\begin{aligned}
 &\text{原宠} \xrightarrow{\text{转生或保留}} \text{成长档} \xrightarrow{\text{融合}} \text{蛋基础成长 } b \\
 &\quad \quad \quad \xrightarrow{\text{喂药与孵化}} \text{孵化后成长 } h \xrightarrow{\text{出生与升级随机}} \text{高等级面板}.
 \end{aligned} \tag{4.28}$$

这一流水线将散落在不同源码函数中的机制统一为状态变换序列。各环节共同呈现出“先截断、再裁切、再按比例分配”的离散结构，因此宠物培养问题天然兼具离散优化与状态转移两重性质。

第五章 宠物养成到战斗能力的映射

本章位于宠物部分的收束位置，其目的在于将前两章得到的成长模型与养成链条联结为一条从“养成输入”到“战斗输出”的完整路径。分析重点在于说明成长率如何转化为攻击、防御与敏捷，转生、融合和喂药如何改变这种转化，以及这些变化为何足以在进入具体战斗公式之前就决定宠物培养的主要方向。

5.1 从 (V, S, T, D) 到战斗属性的映射

宠物成长率并不会直接表现为玩家可见的战斗面板，而是要经由出生随机、升级随机与显示公式的共同作用，最终形成攻击、防御与敏捷。若把高等级时的原始累计点向量记为

$$\mathbf{r} = (V_r, S_r, T_r, D_r), \quad (5.1)$$

则第 3 章已经给出显示映射：生命主要由 V_r 主导，攻击主要由 S_r 主导，防御主要由 T_r 主导，敏捷则近乎由 D_r 单独决定。因此，从边际影响上看，四项成长对战斗能力的主要路径可以概括为

$$V \rightarrow HP, \quad S \rightarrow ATK, \quad T \rightarrow DEF, \quad D \rightarrow SPD. \quad (5.2)$$

式 (5.2) 只刻画主路径。显示公式中仍存在交叉项： V 和 D 会少量进入攻击、防御， S 和 T 也会通过总和与随机路径间接影响成长评分。因而，玩家所谓“主 V 宠”“主 D 宠”，在源码层面更准确的说法是“某一维在主路径上占优势”，而非其余三维完全不起作用。

进入战斗后，显示属性又会进一步转换成固定属性和战斗属性。其中最关键的桥梁变量是敏捷：宠物显示敏捷越高，`WORKFIXDEX` 和 `WORKQUICK` 越容易被抬高，从而在乱敏、回避、会心和反击中占据优势。因此，成长中的 D 维沿着一条较长的链条影响先手权与概率判定，其意义远不止对应面板上增加几点敏捷。

5.2 从转生、融合到战斗能力的长期收益

转生、融合和蛋喂药均作用于“进入成长流程之前的成长档”，而非直接修改战斗公式的输入，因此其收益具有显著的长期累积性。设原始成长档为 g ，转生后成长档为

$g^{(new)}$ ，融合蛋基础成长为 b ，喂药并孵化后的成长档为 h ，则战斗能力的比较实质上是在比较

$$\Psi(g), \quad \Psi(g^{(new)}), \quad \Psi(h), \quad (5.3)$$

其中 $\Psi(\cdot)$ 表示“成长随机 $\rightarrow$ 高等级显示属性 $\rightarrow$ 战斗固定属性 $\rightarrow$ 战斗结果”的复合映射。

转生的长期收益主要体现在两个方面。第一，它通过转生能力值和按权重重分配，整体改变了四维成长的起点；第二，转生后的新成长率进入的是“转生宠 Rank 表”，从而改变了后续升级随机的区间结构。因此，转生同时改变总和、方向和 Rank 解释规则。

融合和喂药的长期收益则更接近一个带约束的优化问题。融合先通过主宠 0.6 与副宠平均后 0.4 生成蛋基础成长，喂药再经过单项折算、50 压缩、按比例回分配和 150 压缩，最后得到孵化后成长 h 。因此，融合链条的长期收益不仅取决于“投入了多少高质量素材”，还取决于这些素材和药效在多次取整与上限约束之后保留了多少。这种离散损耗正是玩家直觉与实际结果发生偏离的重要来源。

5.3 从蛋喂养到战斗目标的定向优化

喂药和孵化在进入普通成长流程之前确定一组新的成长档，因此，对战斗目标的优化首先应落在 h 上。若把一个战斗目标写成效用函数 U ，则蛋喂养问题可抽象为

$$\max_{\text{喂药方案}} U(\Psi(h)). \quad (5.4)$$

在实际工具中，为了降低求解复杂度，常先用一个线性代理目标

$$U_0(h) = w_V h_V + w_S h_S + w_T h_T + w_D h_D \quad (5.5)$$

来近似真正的战斗目标。

式 (5.5) 的意义在于，它将复杂的战斗诉求先投影回孵化后成长空间。例如，若目标偏向先手与回避，则 w_D 应更高；若目标偏向骑宠近战爆发，则 w_S 与 w_D 通常应同时抬高；若目标偏向承伤与稳定性，则 w_V 与 w_T 更为重要。得到候选解之后，再将其带回更完整的战斗模型中作二次比较。

喂药阶段与最终战斗结果之间还隔着出生后随机和升级随机，若直接对最终战斗结果做全局枚举，计算代价会显著升高。对 h 做代理优化，则可以把这一高维随机问题拆成“先确定方向，再在随机层验证”的两步问题。

5.4 宠物忠诚失控 (nono) 与动作改写

宠物成长除了通过面板三围进入战斗外，忠诚还会通过 CHAR_WORKFIXAI 直接改变宠物最终执行的动作。核心函数 BATTLE_PetLoyalCheck() 在每只单位正式执行动作之前调用，位于“动作写入之后、动作分发之前”的主流程分支。

因此，“宠物成长 → 战斗输出”的链条不能只写成属性映射。对低忠诚宠而言，战斗输出更准确地说是“原指令 → 忠诚检查 → 实际动作”的复合结果。

5.4.1 忠诚阈值与改写结果

设当前回合读取到的忠诚值为

$$a = \text{CHAR_WORKFIXAI}, \quad (5.6)$$

并记 $\text{RAND}(1, 100)$ 的结果为 R 。源码不是只设一个“是否 nono”的单阈值，而是按 a 的区间，把宠物分到不同改写模式，如表 5.1 所示。

表 5.1: 宠物忠诚失控 (nono) 的动作改写区间

忠诚区间 a	触发条件	本回合结果
$a \geq 80$	恒不触发	完全按玩家原指令行动
$70 \leq a < 80$	$R < 10$	9% 概率进入“乱目标”
$60 \leq a < 70$	$R < 20$	19% 概率进入“乱目标”
$50 \leq a < 60$	$R < 35$	34% 概率进入“乱目标”
$40 \leq a < 50$	$R < 50$	49% 概率进入“乱目标”
$30 \leq a < 40$	$R < 70$	69% 概率改放另一项宠技
$20 \leq a < 30$	$R < 70$	69% 概率改放另一项宠技
$10 \leq a < 20$	$R < 80$	79% 概率攻击主人，否则攻击敌人
$0 \leq a < 10$	$R < 60$	59% 概率攻击主人，否则直接逃跑

表 5.1 中的概率之所以是 9%、19%、34% 而不是看上去更直观的 10%、20%、35%，原因在于源码使用的是严格不等式 $<$ ，并且随机值来自闭区间整数 $\text{RAND}(1, 100)$ 。例如“ $R < 10$ ”只会在 $R = 1, \dots, 9$ 时成立。

这里还需强调，只有 $20 \leq a < 40$ 这一段的“随机动作”模式，才会把玩家原本指定的宠技 A 改写成另一项宠技 O； $40 \leq a < 80$ 只会打乱目标， $0 \leq a < 20$ 则会进一步退化成为攻击主人、攻击敌人或逃跑。因此，玩家所谓“明明点了 A，结果放成另一项技能 O”的 nono 现象，主要集中在 $20 \leq a < 40$ 这一区间。

5.4.2 为何会出现“技能 A 的面板修正保留，但实际放出技能 O”

这一现象源于源码中的执行顺序。宠物技能在玩家输入阶段就会先写入一组本回合工作值，例如

$$(\text{WORKBATTLECOM1}, \text{WORKBATTLECOM2}, \text{WORKBATTLECOM3}, \\ \text{WORKATTACKPOWER}, \text{WORKDEFENCEPOWER}, \text{WORKQUICK}, \dots). \quad (5.7)$$

随后在战斗主循环中，系统先做状态序列与基础修正，再调用 `BATTLE_PetLoyalCheck()` 检查忠诚；只有在这一步之后，才按 `WORKBATTLECOM1` 进入具体动作分支。于是，若宠物先按玩家原指令 `A` 写入了攻、防、敏等工作值，而忠诚检查又把动作编号改成另一项技能 `O`，那么是否出现“先吃到 `A` 的面板效果，再放出 `O`”，就取决于 `O` 会不会把这些工作值重新覆盖。

源码里这类情况确实存在。例如 `PETSKILL_AttackCrazed()` 会先把

$$\text{WORKATTACKPOWER} \leftarrow 0.8 \text{WORKFIXSTR}, \quad \text{WORKDEFENCEPOWER} \leftarrow 0.7 \text{WORKFIXTOUGH}, \quad (5.8)$$

再写入 `BATTLE_COM_S_ATTICRAZED`；而 `PETSKILL_AttackShoot()` 在当前源码中只改写动作编号、目标和 `WORKBATTLECOM3`，其原本的攻防重写代码被整体注释掉了。于是，若宠物先选中了前者，但在忠诚失控的“随机动作”分支里又被改写成后者，就会出现“前一个技能留下的攻防修正仍在，最终放出的却是后一个技能”的连锁效果。

因此，宠物战斗输出不能只写成“技能 `O` 的伤害公式”。对低忠诚宠，更准确的表达应为

$$\text{最终输出} = \Omega(\text{原指令 } A, \text{忠诚区间}, \text{改写结果 } O, O \text{ 是否覆盖 } A \text{ 已写入的工作值}). \quad (5.9)$$

低忠诚宠的实战收益不仅由成长与技能池决定，还由“本回合动作写入顺序”和“各技能对工作槽的覆盖范围”共同决定。

5.5 不同培养目标下的宠物类型

若把战斗目标按主导维度划分，可以把宠物大致分成四类。第一类是先手/控制型，核心在于尽可能抬高 D ，使宠物本体或骑宠混合后的速度敏更容易跨过乱敏阈值。第二类是近战爆发型，更重视 S 与 D 的组合： S 负责提高攻击路径， D 负责抢先手和提高会心、命中相关概率。第三类是承伤稳定型，其重点通常落在 V 与 T ，以换取更高血量和防御。第四类则是均衡型，在总和受限时尝试避免某一维过高导致整体压缩。

上述分类是不同目标函数下的最优方向，而非彼此独立的自然种类。例如，同样一只高 D 宠，在“骑宠近战白狼”体系中可能被视为先手核心；而在“骑宠远程”体系中，它的价值则可能部分让位于人物本体速度。换言之，宠物类型由人物职业、武器、骑宠状态和战斗目标共同定义。

5.6 人物与宠物协同

人物与宠物协同最典型的体现，就是骑宠状态下的人宠混合公式。后文关于骑宠伤害的章节将进一步说明：骑宠近战时，攻击和速度都更偏宠物；骑宠远程时，攻击和速度则更偏人物。因此，宠物培养的最优方向必须与人物打法协同设计。

若人物走骑宠近战，则宠物的 S 、 T 、 D 都会较深地进入攻击、防御与速度路径，此时高成长宠物的收益往往能被完整兑现。若人物走骑宠远程，则宠物攻击只以较低权重并入人物攻击，而速度也更多取人物本体，因而宠物在 D 维和 S 维上的边际收益会相对下降。类似地，若人物更依赖职业法术控场，则宠物提供的意义可能不在纯爆发，而在于补足先手、承伤或战术容错。

从统一建模角度看，宠物培养目标是最大化人宠组合的总效用

$$U_{\text{team}} = U_{\text{char}} + U_{\text{pet}} + U_{\text{ride}} + U_{\text{synergy}}. \quad (5.10)$$

其中 U_{synergy} 是单独看人物或宠物时最容易遗漏、但在实战中最关键的部分。

第六章 离散优化模型、案例与策略含义

前文关于成长档、转生、融合、喂药与孵化的分析，共同构成了一个高度离散、带多重上限且包含大量取整跳点的优化问题。本章在进入战斗机制的细分分析之前，先将宠物培养与战斗目标统一写成可计算问题，并在案例分析基础上说明这些结果对实际培养与构筑策略意味着什么。

6.1 优化问题的形式化

将整个宠物培养过程写成统一映射后，可以把最终优化问题抽象为

$$\max_{x \in \mathcal{X}} U(\Psi(x)), \quad (6.1)$$

其中 x 表示决策变量， $\mathcal{X}$ 表示所有可行培养方案， Ψ 表示“培养方案 $\rightarrow$ 成长档 $\rightarrow$ 随机成长 $\rightarrow$ 战斗属性 $\rightarrow$ 战斗表现”的复合映射， U 则表示研究者真正关心的目标函数。

在当前工具中， x 至少可以包含三类变量：转生输入、融合输入和喂药方案。它们之中，喂药方案最适合先被形式化为整数规划。若记第 i 维第 ℓ 级药的颗数为 $c_{i\ell}$ ，则其约束满足

$$c_{i\ell} \in \mathbb{Z}_{\geq 0}, \quad \sum_{i=1}^4 \sum_{\ell=1}^5 c_{i\ell} = 40. \quad (6.2)$$

但由于后续还存在折算、单项上限、总和压缩和比例回分配，式 (6.2) 只是最外层约束，而不是完整可行域描述。

6.2 蛋喂养最优化

在喂药问题中，最自然的目标是直接最大化某个最终战斗指标，但这会将离散喂药、孵化、出生随机、升级随机和战斗公式全部卷入同一个搜索中，复杂度极高。因此，当前项目采用两层方法：先在孵化后成长 $\mathbf{h}$ 上做离散优化，再把候选解带回更完整的战斗模型中验证。设蛋基础成长为 $\mathbf{b}$ ，喂药方案为 c ，则喂药层的中间问题可写为

$$\max_c U_0(\mathbf{h}(c; \mathbf{b})), \quad (6.3)$$

其中 U_0 可以是线性目标，也可以是词典序目标。

这一层必须显式建模，因为“全部喂 5 级药”并非总是最优。若药效在折算后已触发 50 压缩或最终 150 压缩，继续把药往同一维堆高只会把别的维度一并拉入压缩区，不再提高目标维度的最终收益。在某些高基础蛋上，混入较低等级药反而可能比全部使用五级药更优。这一现象并非经验技巧，而是由多次向下取整和总和上限共同造成的离散最优化结果。

当前项目还引入了词典序目标，即先最大化主目标，再在主目标并列解中最大化次目标。这一步的必要性来自大量退化解。例如，在药效原始值满足线性关系的情况下，不同药级组合可能给出完全相同的折算结果。若没有次级目标或其它偏好规则，输出候选解就会高度冗余。当前实现偏向更高等级素材，本质上是在这些退化解之间加入一个稳定的优先级。

6.3 战斗目标反推

真正的战斗目标往往不是“让某个孵化后成长最大”，而是“让先手概率、命中率、会心率或骑宠伤害最大”。因此，需要将战斗目标反投影回成长空间。若记战斗目标为

$$U_{\text{battle}} = U(HP, ATK, DEF, SPD, p_{\text{hit}}, p_{\text{crit}}, \text{damage}, \dots), \quad (6.4)$$

则一个常见策略是先在当前点做局部线性化，把它近似为

$$U_{\text{battle}} \approx w_V h_V + w_S h_S + w_T h_T + w_D h_D + \text{const}, \quad (6.5)$$

再回到式 (6.3) 求解。

例如，若目标是提升玩家对玩家的先手与会心，则由于这两项都对敏捷高度敏感，线性化后通常会给 w_D 更高权重；若目标是骑宠近战爆发，则 w_S 与 w_D 往往都会较高，因为 S 主要影响攻击，而 D 同时影响先手和会心；若目标是承伤，则应提高 w_V 与 w_T 。这一“战斗目标 $\rightarrow$ 成长代理目标”的反推步骤，构成了工具中各种喂药搜索模式背后的统一数学结构。待后文把行动顺序、概率链、装备与场地因素纳入分析后，局部线性化对象还可以进一步扩展成“培养方案 $\rightarrow$ 完整战斗决策收益”的多层代理目标。

6.4 转生与融合策略比较

转生与融合属于不同层次的决策。前者重塑成长档后再交给完整成长随机，后者则把多只宠物的成长信息压缩进一个蛋面中间态。因此，两者面临的约束瓶颈并不相同。

转生的主要瓶颈来自转生能力上限 A_{max} ，高端区间本质上是方向重分配问题。融合的主要瓶颈则来自逐层取整：低等级宠先做 0.8 折扣，再参与主副权重分配，最后蛋又要经过孵化压缩，损失沿途积累。

若当前已有高总和且方向一致的成熟宠，转生更适合“保留方向并整体抬高”；若目标是把多只宠物的局部优势重新拼装，则融合与喂药更适合“重构方向”。但一旦进入压缩区，无论转生还是融合，都不再适合用“总和多一点一定更好”的线性直觉判断。

6.5 案例研究

下面给出三个不依赖具体服务器数值、但足以说明机制特征的案例。

6.5.1 案例一：先手优先的骑宠近战

设目标是让一只骑宠近战配置在 PVP 中尽量抢到先手，同时保留一定爆发。结合后文关于行动顺序与骑宠伤害的分析可知，骑宠近战时速度参照与攻击都更偏宠物，因此代理目标应更偏向 D 与 S 。一种自然的近似目标是

$$U_1 = \alpha h_D + \beta h_S, \quad \alpha > \beta > 0. \quad (6.6)$$

若基础蛋本来就有较高总和，则继续把所有高等级药堆到 D 上，可能因为触发总和压缩而连带损失 S ，从而在最终骑宠伤害上得不偿失。这正是词典序目标或多候选解比较有价值的场景。

6.5.2 案例二：高基础蛋的压缩陷阱

设一枚蛋的基础成长总和已经接近最终上限。若继续用“每一颗药都尽量给最高级”的策略，则折算后的工作值很容易触发 50 压缩，而孵化后成长又可能触发 150 压缩。此时，药效的边际收益不再等于原始药值。对这类问题，最优方案常常不是单纯堆单项，而是控制进入压缩前的分布，使目标维度在压缩后仍保留最大整数份额。离散最优解更接近“卡阈值”而非“无限堆高”。

6.5.3 案例三：主目标并列下的次目标选择

设若干喂药方案都能把主目标维度推到同一上限，此时单看主目标已经无法区分优劣。若没有次目标或偏好规则，输出会出现大量本质等价的退化解。当前项目采用“主目标 + 次目标 + 高等级素材优先”的方案，本质上是在一个大并列集合里引入稳定排序。这样做的意义不是改变底层规则，而是让结果更适合人类使用和二次微调。

综合以上案例可以看到，底层机制本身是一个离散、分段、带上限的系统，适合沿源码结构拆成可验证的中间层，再逐层向战斗目标推进。工具实现可先以孵化后成长、再以高等级面板、最后以战斗代理收益三层逐步接入；后续研究也可以在此基础上扩展蒙特卡洛实验、PVP/PVE 参数扫描以及多目标求解。

第七章 战斗有效属性与状态变量

后续各章对行动顺序、命中概率、法术抗性与伤害层次的分析，都依赖于一组统一的战斗变量口径。本章据此整理人物、宠物与敌方单位在战斗中的有效属性层次、身份差异以及关键状态槽的更新方式，以避免将不同读写阶段的变量误写为同一量。

7.1 有效属性的三层口径

战斗中存在三层属性口径。原始属性（如 CHAR_STR、CHAR_DEX、CHAR_LUCK）是回合开始前的基础值。固定属性（如 CHAR_WORKFIXSTR、CHAR_WORKFIXDEX、CHAR_WORKFIXLUCK）是经装备、套装和战斗初始化结算后的当前回合工作值。战斗属性（如 CHAR_WORKATTACKPOWER、CHAR_WORKDEFENCEPOWER、CHAR_WORKQUICK）是在固定属性上再经本回合攻防速修正后的最终判定量。

把这一过程抽象写成函数链，可以记为

$$X_{\text{raw}} \xrightarrow{\mathcal{F}} X_{\text{fix}} \xrightarrow{\mathcal{T}} X_{\text{battle}}, \quad (7.1)$$

其中 $\mathcal{F}$ 表示装备、状态和战斗初始化引起的固定化过程， $\mathcal{T}$ 表示 BATTLE_TurnParam() 这一类“按本回合修正生成攻、防、速工作值”的过程。以敏捷为例，更具体的关系可以理解为

$$\text{DEX}_{\text{raw}} \longrightarrow \text{DEX}_{\text{fix}} = \text{WORKFIXDEX} \longrightarrow Q = \text{WORKQUICK}. \quad (7.2)$$

行动顺序主要读取 Q ，物理回避、会心、反击则读取 DEX_{fix} 。

幸运的层级尤其容易混淆：物理回避、会心与反击读取固定幸运 CHAR_WORKFIXLUCK，而法术命中读取原始幸运 CHAR_LUCK。这意味着物理系统与法术系统并不共享同一层幸运变量。

7.2 单位身份与公式切换

战斗公式中许多修正由单位身份（人物、宠物、敌方单位）而非技能决定。不同身份组合通过不同缩放系数进入同一条概率公式。

当前公式中的身份差异主要分两类。敏捷缩放差异：根据攻守双方身份，对某一方敏捷先乘 0.8 或 0.6 再截断，然后进入平方根结构。替代变量差异：法术目标为非玩家时，系统不再读取幸运与抗魔装备，改用等级近似。

因此，在“人物对人物”口径下得到的命中、回避与会心结论，不能直接外推到其他身份组合。后文将据此把人物、宠物与骑宠三类口径分别处理。

7.3 关键状态变量及其作用入口

战斗状态分散在 CHAR_WORK*、CHAR_WORKBATTLEFLG、CHAR_WORKBATTLECOM* 和战场全局结构中。以下列出对后文最关键的变量。

首先是若干会直接改变判定入口的布尔标记。防御动作 BATTLE_COM_GUARD 会让普通回避在最前面直接失败；CHAR_BATTLEFLG_NODUCK 表示“不能闪避”；CHAR_BATTLEFLG_REVERSE 则表示单位当前处于属性反转状态。其次是若干会改变具体参数的工作变量，例如 CHAR_WORKBATTLECOM3 会被当前动作复用，高 16 位和低 16 位在不同技能中可分别表示技能等级、子类型、阵列或额外修正值；CHAR_MYSKILLHIT 与 CHAR_MYSKILLHIT_NUM 则对应某些“提高命中”类主动状态。

与宠物相关的 CHAR_BATTLEFLG_AIBAD（即玩家常称的“nono / 忠诚失控”标记）在 BATTLE_PetLoyalCheck() 中按回合清空后重新判断，表示该宠物本回合未按玩家原指令行动，进一步影响反击和同队误伤保护等分支。

回避系统中还需区分两套同名但进入位置不同的状态。_PETSKILL_SETDUCK 相关的宠技回避状态借助 CHAR_MYSKILLDUCK（剩余回合）与 CHAR_MYSKILLDUCKPOWER（独立前判概率），前判成功则整次物理攻击直接判定为闪避。职业技能“回避”对应的 CHAR_WORK_P_DUCK（启用标记）与 CHAR_WORKMOD_P_DUCK（倍率值），则在普通回避值算出后做乘法放大，不生成独立闪避事件。

属性系统相关的状态槽包括四属性结界，分别写在 CHAR_WORKFIXEARTHAT_BOUNDARY、CHAR_WORKFIXWATERAT_BOUNDARY、CHAR_WORKFIXFIREAT_BOUNDARY、CHAR_WORKFIXWINDAT_BOUNDARY；属性抗性写入 CHAR_WORK_F_RESIST、CHAR_WORK_I_RESIST、CHAR_WORK_T_RESIST；场地属性、场地强度与回合数存放在战场级结构 BattleArray[battleindex] 中。由此可见，属性系统并不是单一状态，而是一组作用层级与刷新逻辑均不相同的子状态集合。

7.4 状态槽的存储方式与回合更新

战斗状态的存储分三种。单纯标志位（如 NODUCK、REVERSE）通过位运算开关。整数槽位（如 WORKFIXDEX、WORKQUICK、各类抗性）在每回合预处理时重新写入。高低位打包状态，典型代表为四属性结界使用的

$$\text{MAKE2VALUE}(\text{power}, \text{turn}), \quad (7.3)$$

它用高 16 位存强度、低 16 位存剩余回合。

状态的时间更新方式各异：场地（战场级）每回合 `att_count` 减一，减到非正时清空；四属性结界（单位级）每回合低 16 位回合数减一，减到 `-1` 时清空；属性反转不自动衰减，靠技能再次命中时异或切换。

这些状态分别进入预处理、概率判定、属性修正与回合衰减等不同阶段，因此名义上相近的“减伤”或“降敏”，在源码中可能对应完全不同的机制。后文所有公式均以前述变量层次与更新时序为前提。

第八章 行动顺序与速度分支结构

行动顺序并非由动作中文名直接决定，而是由动作在 `BATTLE_DexCalc()` 中进入何种速度分支决定。围绕该问题，经验口径中最常见的混淆主要有两类：其一，哪些动作读取骑宠混合后的参照敏捷，哪些动作只读取人物本体 `WORKQUICK`；其二，哪些动作适用普通乱敏结构，哪些动作还带有额外加速项或专用速度档。本章据此按源码分支而非玩家习惯称呼重构行动顺序公式。

8.1 参照敏捷的来源与骑宠权重

设人物的战斗速度敏为 $Q_c = \text{WORKQUICK}$ ，骑宠的战斗速度敏为 Q_p 。若动作走默认速度分支，则当前编译口径下的参照敏捷可写为

$$Q_{\text{ref}} = \begin{cases} Q_c, & \text{未骑宠,} \\ 0.8Q_c + 0.2Q_p, & \text{骑宠且装备远程武器,} \\ 0.2Q_c + 0.8Q_p, & \text{骑宠且装备近战武器或空手.} \end{cases} \quad (8.1)$$

这里的“远程武器”并不只包括弓，还包括回力类与投掷类；源码通过 `BATTLE_IsThrowWepon()` 将这些武器归为同一类。式 (8.1) 表明，骑宠远程仍以人物速度为主，而骑宠近战则改由宠物速度主导。

因此，骑宠对行动顺序的影响首先不是统一加速，而是改写参照敏捷的来源。只有最终进入默认速度分支的动作，才会读取式 (8.1) 的混合结果；若动作转入只读取人物本体 `WORKQUICK` 的专用分支，则骑宠速度不会参与该动作的先手计算。

8.2 动作分支与顺序公式

以下分别整理普通攻击、普通物品、普通法术 / 精灵 / 带法术字段的物品以及急速类动作的速度公式。关键不在动作名称，而在于其最终进入的源码分支。

普通攻击与默认分支

若动作走默认分支，则先取

$$w = Q_{\text{ref}} + 20, \quad (8.2)$$

再生成顺序值

$$S_{\text{normal}} = w - \text{RAND}(0, 0.3w). \quad (8.3)$$

式 (8.3) 即玩家通常所称的“乱敏”。于是有

$$S_{\text{normal}} \in [0.7w, w]. \quad (8.4)$$

人物普通攻击、宠物普通攻击，以及大量沿用默认速度档的职业技能，都属于这一组。此时若人物处于骑宠状态，式 (8.1) 就会直接改写其先手顺序。

使用普通物品

普通物品使用走 BATTLE_COM_ITEM 分支。它与默认分支的关键差别有两点：第一，参照速度不再取骑宠混合值，而是直接取人物本体 WORKQUICK；第二，在普通乱敏基础上额外附带 15% 的顺序加项。其公式可整理为

$$w_{\text{item}} = Q_c + 20, \quad (8.5)$$

$$S_{\text{item}} = w_{\text{item}} - \text{RAND}(0, 0.3w_{\text{item}}) + 0.15w_{\text{item}}. \quad (8.6)$$

因此，人物使用普通物品时只计算人物本体速度敏，而不计算骑宠混合后的调整敏捷。

人物释放精灵、施放普通法术与使用带法术字段的物品

人物施放普通法术、使用精灵，以及使用带法术字段的物品时，会统一进入 BATTLE_COM_JYUJYUTU 分支，而不是普通物品分支。这里所说的“带法术字段的物品”需要按源码继续区分：其一，是显式挂接 ITEM_useMagic 的直接施法道具；在当前物品数据中，此类条目可确认者仅有“大地之怒”一例。其二，是带有有效 ITEM_MAGICID 的装备或少量特殊物品，它们在客户端可被识别为可用法术，并在战斗中同样落入该分支。因此，本节所讨论的不是一个狭义的玩家口语类别“魔法类道具”，而是一组在实现上共享同一速度公式的动作来源。其速度公式为

$$w_{\text{spell}} = Q_c + 20, \quad (8.7)$$

$$S_{\text{spell}} = w_{\text{spell}} - \text{RAND}(0, 0.3w_{\text{spell}}). \quad (8.8)$$

这一分支与普通物品的共同点，在于同样只读取人物本体 WORKQUICK；不同点则在于它没有式 (8.6) 中额外的 15% 顺序加成。

因此，人物释放精灵和使用普通物品都不读取骑宠混合敏；两者的差别在于，普通物品额外快 15%，而精灵、普通法术以及上述带法术字段的物品没有这一项。

急速攻击

急速攻击类动作走专用加速分支。当前实现下其公式是

$$w_{\text{quick}} = Q_c + 20, \quad (8.9)$$

$$S_{\text{quick}} = 1.3 w_{\text{quick}}. \quad (8.10)$$

这表明，急速攻击的实现口径是“先 +20，再乘 1.3”，而不是“直接在原始 WORKQUICK 上乘 1.3”。与此同时，这一分支同样只读取人物本体 WORKQUICK，不读取骑宠混合后的参照敏捷。

8.3 职业技能的速度口径

职业技能是否受骑宠影响，并不由“白狼 / 猎人 / 法师”这三个职业标签本身决定，而是由技能最终落入哪一条速度分支决定。就源码结构而言，可归纳为以下三类。

8.3.1 沿用默认分支的职业技能

若职业技能最终仍走默认速度分支，则其顺序公式仍是式 (8.3)，只是其中的参照敏捷改为式 (8.1) 的 Q_{ref} 。这意味着：若人物没有骑宠，则直接读人物速度敏；若人物处于骑宠状态，则近战 / 远程武器类型与骑宠敏捷会共同影响先手。

白狼系中大量主动攻击与辅助技能属于这一类，例如爆击、连环攻击、双重攻击、盾击、格挡、贯穿攻击、回旋攻击、混乱攻击等。猎人系中大量技能也属于这一类，例如陷阱、驯伏宠物、激怒宠物、天罗地网、树根缠绕、体力掠夺、毒素武器、弱点攻击、挑拨、遗忘等。

因此，盾击、天罗地网这类职业技能在当前实现下通常会受骑宠混合敏影响；只要它们落在默认分支，骑宠就会改变其先手性质。

8.3.2 只读人物本体速度的职业技能

另一类职业技能虽然属于职业技能，但并不走默认分支，而是直接把人物本体 WORKQUICK 写入专用速度公式。法师系技能最典型。令

$$w_m = Q_c + 20, \quad (8.11)$$

则当前源码中的典型速度档包括：

表 8.1: 法师系技能的主要速度档

分组	速度公式	代表技能
快法术档	$w_m - \text{RAND}(0, 0.2w_m)$	火山泉、召雷术、冰箭术
普通法术档	$w_m - \text{RAND}(0, 0.3w_m)$	嗜血成性、嗜血蛊、一针见血
封印档	$w_m - \text{RAND}(0.3w_m, 0.5w_m)$	火附体、冰附体、雷附体
重法术档	$w_m - \text{RAND}(0, 0.5w_m)$	火星球、电流术、冰爆术
延迟重法档	$w_m - \text{RAND}(0.2w_m, 0.5w_m)$	火龙枪、暴风雨、冰镜术、附身术、移形换位

其中“世界末日”与“火龙枪”还带有延迟释放特性，因此“读到指令的回合”与“真正结算的回合”并不相同。就行动顺序而言，这类技能的共同点仍是只看人物本体 WORKQUICK，而不看骑宠混合敏。

8.3.3 专用加速职业技能

少数职业技能还会进入额外的专用速度档，例如前节的急速攻击就属于“先 $Q_c + 20$ ，再整体乘上固定倍率”的类型。对这类技能，是否受骑宠影响的判断方法与前两类相同：只要公式直接从人物本体 WORKQUICK 出发，而不是从 Q_{ref} 出发，骑宠速度就不会参与该技能的先手计算。

8.4 先手区间与阈值判断

把乱敏写成区间模型后，很多先手结论可以直接由支撑区间比较得到。若双方动作都走默认分支，分别记

$$w_a = Q_{\text{ref},a} + 20, \quad w_b = Q_{\text{ref},b} + 20, \quad (8.12)$$

则两边的顺序值支撑区间为

$$S_a \in [0.7w_a, w_a], \quad S_b \in [0.7w_b, w_b]. \quad (8.13)$$

于是立刻得到三类判断：

$$0.7w_a > w_b \implies a \text{ 必定先手}, \quad (8.14)$$

$$w_a < 0.7w_b \implies a \text{ 不可能先手}, \quad (8.15)$$

其余情形则属于区间重叠区，需要继续做离散随机比较。

这说明先手收益具有明显的阈值结构。在远离阈值时，少量敏捷提升对先手结果的影响有限；一旦逼近由 0.7 与 1.0 构成的边界，少量速度变化就可能显著改变先手概率。骑宠近战与骑宠远程由于改写了 Q_{ref} 的来源，也会同步平移这条先手阈值曲线。

若要比物品、精灵与普通攻击之间的先手，还必须先区分公式所属分支：普通攻击通常使用式 (8.3)，普通物品使用式 (8.6)，精灵 / 普通法术 / 带法术字段的物品使用式 (8.8)，急速攻击使用式 (8.10)。这四类动作不能共用同一条“乱敏直觉”。

默认乱敏规则下的先手区间判别表

	80	120	160	200	240	280	320
80	重叠	不先	不先	不先	不先	不先	不先
120	必先	重叠	重叠	不先	不先	不先	不先
160	必先	重叠	重叠	重叠	不先	不先	不先
200	必先	必先	重叠	重叠	重叠	重叠	不先
240	必先	必先	必先	重叠	重叠	重叠	重叠
280	必先	必先	必先	重叠	重叠	重叠	重叠
320	必先	必先	必先	必先	重叠	重叠	重叠

行: 攻方参照敏 + 20 ; 列: 守方参照敏 + 20

图 8.1: 默认乱敏规则下的先手区间判别表。图中以 $w_a = Q_{\text{ref},a} + 20$ 、 $w_b = Q_{\text{ref},b} + 20$ 为行列，Always 表示攻方必定先手，Never 表示攻方不可能先手，Overlap 表示双方顺序值区间重叠，需进一步做离散随机比较。

第九章 物理命中、回避、会心与反击

本章讨论普通物理链中的四类核心概率：命中、回避、会心与反击。相较于经验总结，源码实现更强调判定层次、对象身份和离散截断次序；同一字段在不同链路中的进入位置不同，其真实收益也因而不同。以下分析均以源码读写顺序为最高证据口径。

9.1 普通回避链的判定分层

当前实现中，防守方的闪避判定依次经过三层：硬性失败条件、独立前判，以及普通回避主体。若把最终总闪避率记为 p_{total} ，则最先需要区分的不是主体公式本身，而是“是否存在独立前判”，因为这会决定后续概率是串联还是单段。

普通回避的硬性失败条件包括：当前动作是 `BATTLE_COM_GUARD`¹、存在吸收/反射/消失类伤害反应、当前无法行动且不满足“集气但未被天罗地网或盾击限制”的例外、挂有 `CHAR_BATTLEFLG_NODUCK`，以及特殊战斗标记。任一条件成立即跳过普通回避公式。

若编译启用 `_PETSKILL_SETDUCK`²，防守方可能在普通回避前先经过额外回避前判。该前判的影响因素只有两项：其一是 `CHAR_MYSKILLDUCK`，用于表示前判剩余回合是否仍然存在；其二是 `CHAR_MYSKILLDUCKPOWER`，用于表示前判本身的成功率。若前判不存在，则 $p_{\text{front}} = 0$ ；若前判存在，则先以 p_{front} 独立判定一次，失败后才进入普通回避主体计算 p_{normal} 。用真实技能表述时，法师主动技能“移形换位”正是这一层的典型例子：`profession.txt` 将其登记为 `PROFESSION_TRANSPOSE`，而战斗执行阶段会把它落到 `BATTLE_COM_S_TRANSPOSE`，随后直接写入 `CHAR_MYSKILLDUCK` 与 `CHAR_MYSKILLDUCKPOWER`。因此，“移形换位”增加的是独立前判 p_{front} ，而不是普通回避主体内部的倍率项。

定理 9.1 (前判与普通回避的复合). 设前判成功率为 p_{front} ，普通回避主体成功率为 p_{normal} ，则总闪避率为

$$p_{\text{total}} = p_{\text{front}} + (1 - p_{\text{front}})p_{\text{normal}}. \quad (9.1)$$

证明. 先以前判概率 p_{front} 直接闪避；若前判失败，概率为 $1 - p_{\text{front}}$ ，则进入普通回避主体，成功率为 p_{normal} 。两段顺序串接，因此总成功率为两项相加。□

¹`gmsv/src/battle/battle_event.c` 中 `BATTLE_DuckCheck()` 的开头判定，当前版本位于第 763 行附近。

²`gmsv/src/include/version.h`, 第 287 行。

注 9.2. 白狼系职业技能“回避”不是独立前判。PROFESSION_AVOID 在执行后写入的是 CHAR_WORK_P_DUCK 与 CHAR_WORKMOD_P_DUCK，其中前者仅为“启用倍率”标记，后者为倍率值；两者将已算出的普通回避值乘以比例，属于普通回避主体的后段修正，而不是先于主体公式独立判定一次。换言之，若以“移形换位”和白狼“回避”作对照，则前者进入 p_{front} ，后者只改写 p_{normal} 的后段倍率。

把熟练度固定到 100 时，这一区别可以写得更直观。源码中，法师系技能“移形换位”先经 $\text{PROFESSION_CHANGE_SKILL_LEVEL_M}(100)=10$ 换算为技能等级 10，随后写入 $\text{CHAR_MYKILLDUCKPOWER}=70$ ，故此时

$$p_{\text{front}} = 0.70. \quad (9.2)$$

若该角色进入普通回避主体后的成功率为 $p_{\text{normal}} = 0.30$ ，则总闪避率为

$$p_{\text{total}} = 0.70 + (1 - 0.70) \times 0.30 = 0.79. \quad (9.3)$$

也就是说，在这一例子中，79% 的总闪避率里，主导部分来自“移形换位”提供的独立前判，而不是普通回避主体本身。

相对地，白狼系技能“回避”在熟练度 100 时经 $\text{PROFESSION_CHANGE_SKILL_LEVEL_A}(100)=10$ 换算为技能等级 10，再由源码得到

$$P = (10 - 5) \times 3 = 15, \quad (9.4)$$

即把普通回避主体后段放大为原来的 1.15 倍。若放大前的主体判定值为 $u_1 = 3000$ ，对应 $p_{\text{normal}} = 30.00\%$ ，则有

$$u_2 = \left\lfloor \frac{3000(100 + 15)}{100} \right\rfloor = 3450, \quad (9.5)$$

亦即

$$p'_{\text{normal}} = 34.50\%. \quad (9.6)$$

若改以 $u_1 = 6000$ 为例，则放大后为 $u_2 = 6900$ ，对应 69.00%。因此，“回避”带来的并不是新的 p_{front} ，而是对既有 p_{normal} 做比例放大；它的增益大小随普通回避主体原值一同变化。

9.2 回避公式与对象分支

9.2.1 对象身份修正与主判定顺序

设攻击方与防守方原始固定敏分别为 att_{dex} 与 def_{dex} 。进入普通回避主体前，源码按攻守身份做单边修正³，而非对两边同时缩放。记修正后的固定敏为 $\widetilde{att_{\text{dex}}}$ 与 $\widetilde{def_{\text{dex}}}$ ，则在

³gmsv/src/battle/battle_event.c 中 BATTLE_DuckCheck() 的身份分支，当前版本位于第 806-836 行附近。

当前主分支下可整理为

$$\widetilde{(att_{dex}, def_{dex})} = \begin{cases} ([0.8att_{dex}], def_{dex}), & \text{敌人打宠物,} \\ (att_{dex}, [0.8def_{dex}]), & \text{非敌人打宠物,} \\ ([0.6att_{dex}], def_{dex}), & \text{非玩家打玩家,} \\ (att_{dex}, [0.6def_{dex}]), & \text{玩家打非玩家,} \\ (att_{dex}, def_{dex}), & \text{其余情形.} \end{cases} \quad (9.7)$$

所有取整来自源码将浮点乘法结果回写到整型变量的截断。

令

$$B = \max\{\widetilde{att_{dex}}, \widetilde{def_{dex}}\}, \quad S = \min\{\widetilde{att_{dex}}, \widetilde{def_{dex}}\}, \quad (9.8)$$

并定义比例项

$$w = \begin{cases} 1, & \widetilde{def_{dex}} \geq \widetilde{att_{dex}}, \\ \frac{\widetilde{def_{dex}}}{\widetilde{att_{dex}}}, & \widetilde{def_{dex}} < \widetilde{att_{dex}}. \end{cases} \quad (9.9)$$

再定义动作参数

$$\alpha = \begin{cases} 0.027, & \text{若防守方当前动作是 BATTLE_COM_JYUJYUTU (咒术),} \\ 0.02, & \text{否则.} \end{cases} \quad (9.10)$$

则普通回避主体的核心基础值为

$$d_{\text{core}} = \sqrt{\max\left\{\frac{B-S}{\alpha}, 0\right\}} w. \quad (9.11)$$

在此之后，源码按如下顺序继续叠加或修正：

$$d_{\text{pre}} = d_{\text{core}} + L_d + M + R_{\text{drunk}} + 40I_{\text{bow}} + N, \quad (9.12)$$

$$u_0 = \text{clip}_{[1,7500]}(100d_{\text{pre}}), \quad (9.13)$$

$$u_1 = \max\{0, u_0 - \text{RAND}(0.8R_{\text{hit}}, 1.2R_{\text{hit}})\}, \quad (9.14)$$

$$u_2 = \left\lfloor \frac{u_1(100+P)}{100} \right\rfloor, \quad (9.15)$$

$$u_3 = \begin{cases} 1.4u_2, & \text{若攻击方当前使用混乱攻击,} \\ u_2, & \text{否则.} \end{cases} \quad (9.16)$$

最终以 $\text{RAND}(1, 10000) \leq u_3$ ⁴ 判定是否闪避成功。这里 L_d 表示防守方幸运； M 表示全局回避修正 `gBattleDuckModify`； R_{drunk} 表示攻击方醉酒时附加的 $\text{RAND}(20, 30)$ ，否则

⁴`gmsv/src/battle/battle_event.c` 中 `BATTLE_DuckCheck()` 的尾段随机判定，当前版本位于第 893 行附近。

取 0; I_{bow} 为攻击方是否持弓的指示量, 源码该分支额外加两次 20; N 表示 BATTLE_COM_S_NOGUARD 带来的技能附加值; R_{hit} 表示 AddHit 在后段扣掉的命中修正 (源码中该值并非固定值, 而是在 $[0.8R_{\text{hit}}, 1.2R_{\text{hit}}]$ 区间内随机取值, 且仅当攻击方是玩家时生效); P 表示职业技能“回避”的倍率加成。

9.2.2 咒术动作分支与职业技能是否共用回避链

从源码命名与分支用途看, BATTLE_COM_JYUJYUTU⁵ 可译作“咒术动作分支”。该常量名对应日文“咒术”的罗马字写法; 现代常见拼写为 *jujutsu*, 源码保留较旧的 JYUJYUTU 形式。式 (9.17) 中 $\alpha = 0.027$ 的触发条件仅取决于防守方当前动作是否为 BATTLE_COM_JYUJYUTU, 而不取决于攻击方是否使用职业技能。

职业技能下达动作时统一写入各自的 BATTLE_COM_S_*, 不直接标记为 BATTLE_COM_JYUJYUTU。“使用职业技能”与“进入咒术回避参数分支”不是同一件事。需要区分的是: 该职业技能后续是否仍调用普通物理攻击序列 BATTLE_AttackSeq()⁶, 从而继续进入 BATTLE_DuckCheck()⁷。若是, 则仍受“目标当前动作是 BATTLE_COM_JYUJYUTU”影响; 若否, 则不按本章的普通物理回避链解释。表 9.1 汇总当前源码下的关键分类。

⁵gmsv/src/include/battle.h, 第 45 行。

⁶gmsv/src/battle/battle_event.c 中 BATTLE_AttackSeq(), 当前版本位于第 1619 行附近。

⁷gmsv/src/battle/battle_event.c 中 BATTLE_DuckCheck(), 当前版本位于第 763 行附近。

表 9.1: 职业技能与普通物理回避链的关系

技能类别	代表技能	主要结算入口	调用 BATTLE_D uckCheck	是否受 目标 JYUJYUTU 的 0.027 影响
直接物理职业 攻击	爆裂、连环攻击、双 重攻击、骑宠攻击、 舍命攻击、弱点攻 击、掠夺、混乱攻击	battle_professio n_attack_fun() → BATTLE_AttackSeq ()	是	是；但触 发前提仍 是目标当 前动作为 BATTLE_C OM_JYUJY UTU
带物理命中判 定的状态攻击	盾击、剧毒武器	battle_professio n_status_chang_f un() → BATTLE_Att ackSeq()	是	是；逻辑 与直接物 理职业攻 击相同
辅助与自我强 化技能	专注战斗、回避、回 复、舍己为人、激 怒、能量聚集、陷 阱、驯化等	battle_professio n_assist_fun()	否	否；不按 普通物理 回避链处 理
法师攻击技能	火球、冰箭、火枪、 雷击、末日、血蛊等	battle_professio n_attack_magic_f un() → PROFESSION _MAGIC_ATTAC()	否	否；应按 职业法术 命中链分 析
主分发归入法 术攻击组的特 殊技能	穿透攻击、缠绕攻击	当前主分发进入职业 法术攻击组	本文暂不 按普通物 理回避链 归类	本文暂 不把它 们并入 $\alpha = 0.027$ 的普通物 理回避分 析，留待 后续校核

表 9.1 的核心含义在于：职业技能并不会因为“职业技能”这一身份而自动进入 BATTLE_COM_JYUJYUTU 分支；只有那些后续仍调用普通物理回避函数的职业技能，才会在

目标当前动作恰好是咒术分支时, 间接受到 $\alpha = 0.027$ 的影响。

9.2.3 PVP 内部的人宠对象组合差异

即使在玩家对玩家战斗中, 人打人、人打宠、宠打人、宠打宠也不能共用同一简化公式。源码先按攻击侧与防守侧的单位类型进入不同分支, 再决定缩放哪一边的固定敏, 以及是否切到线性口径。表 9.2 汇总当前源码下三类物理概率机制在 PVP 内部的对象组合差异。这里“人”指玩家本体,“宠”指玩家宠物。

表 9.2: PVP 中不同攻守对象组合的概率口径差异

机制	人打人	人打宠	宠打人	宠打宠
回避	无身份缩放; 守方幸运生效	守方敏先截断为 $[0.8def_{dex}]$	攻方敏先截断为 $[0.6att_{dex}]$; 守方幸运生效	守方敏先截断为 $[0.8def_{dex}]$
会心	$\beta = 0.09$, 平方根口径; 攻方幸运与武器会心生效	守方敏先截断为 $[0.6def_{dex}]$; 其余同人打人	改为 $\beta = 10$ 的线性口径; 通常无幸运项、无武器会心项	$\beta = 0.09$, 平方根口径; 通常无幸运项、无武器会心项
反击	$\gamma = 0.08$, 平方根口径; 武器矩阵 F 与幸运生效	守方敏先截断为 $[0.6def_{dex}]$; 仍读武器矩阵 F 与幸运	改为 $\gamma = 10$ 的线性口径; 走宠物反击分支, 不读武器矩阵 F 与幸运, NOGUARD 可加值	$\gamma = 0.08$, 平方根口径; 走宠物反击分支, 不读武器矩阵 F 与幸运, NOGUARD 可加值

其中有两个易错点。第一, 回避判断中“人打宠”与“宠打宠”都命中“防守方是宠物”分支, 因此都将守方固定敏截断到 0.8 倍, 而非常被误写的 0.6 倍。第二, 会心与反击的身份分支不完全等于回避: 当攻击侧是宠物、防守侧是玩家时, 两者都切到“除以 10 且不开根号”的线性口径。

后文给出显式导数和切片图时, 统一固定在“人打人”支路上; 人打宠、宠打人和宠打宠需按表 9.2 先切换到对应口径, 再进入各自的离散计算。

9.2.4 玩家打玩家的离散化表达

若只考虑玩家对玩家, 并忽略额外前判、弓、醉酒、AddHit、职业技能“回避”、无防御攻击附加与混乱攻击修正, 则有 $\widetilde{att_{dex}} = att_{dex}$ 、 $\widetilde{def_{dex}} = def_{dex}$ 。

定义 9.3 (普通回避基础体). 在玩家 PVP 简化条件下, 普通回避基础体定义为

$$d_{\text{base}}(att_{\text{dex}}, def_{\text{dex}}) = \begin{cases} \sqrt{\frac{def_{\text{dex}} - att_{\text{dex}}}{0.02}}, & def_{\text{dex}} \geq att_{\text{dex}}, \\ \sqrt{\frac{att_{\text{dex}} - def_{\text{dex}}}{0.02}} \frac{def_{\text{dex}}}{att_{\text{dex}}}, & def_{\text{dex}} < att_{\text{dex}}. \end{cases} \quad (9.17)$$

定理 9.4 (离散化普通回避率). 在玩家 PVP 简化口径下, 源码对应的普通回避率为离散化结果:

$$p_{\text{duck}}^{\text{src}}(att_{\text{dex}}, def_{\text{dex}}, L) = 0.01 \min\{7500, \max\{1, \lfloor 100(d_{\text{base}}(att_{\text{dex}}, def_{\text{dex}}) + L) \rfloor\}\}. \quad (9.18)$$

普通命中率为

$$p_{\text{hit}}^{\text{src}}(att_{\text{dex}}, def_{\text{dex}}, L) = 100.00 - p_{\text{duck}}^{\text{src}}(att_{\text{dex}}, def_{\text{dex}}, L). \quad (9.19)$$

注 9.5. 式 (9.18) 与式 (9.19) 是当前源码在玩家 PVP 简化条件下的离散百分比结果; 后文做导数分析时, 暂时去掉取整与上下限裁切, 只保留对应的连续化主体。图 9.1 展示回避率在攻守敏捷平面上的分布, 图 9.2 展示命中率随攻方敏捷的变化。

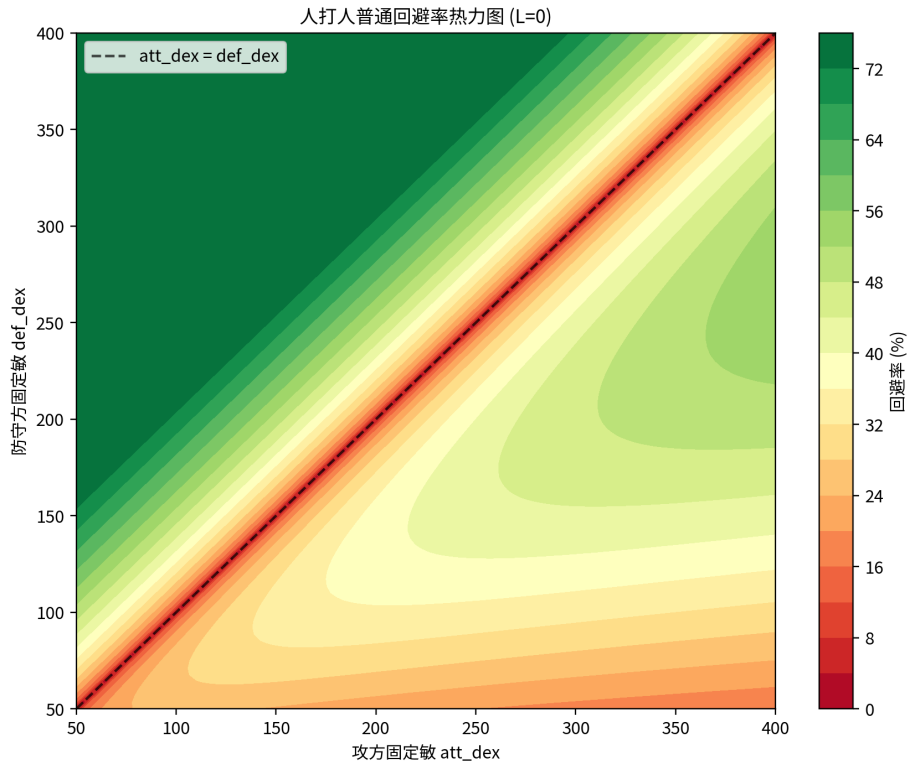
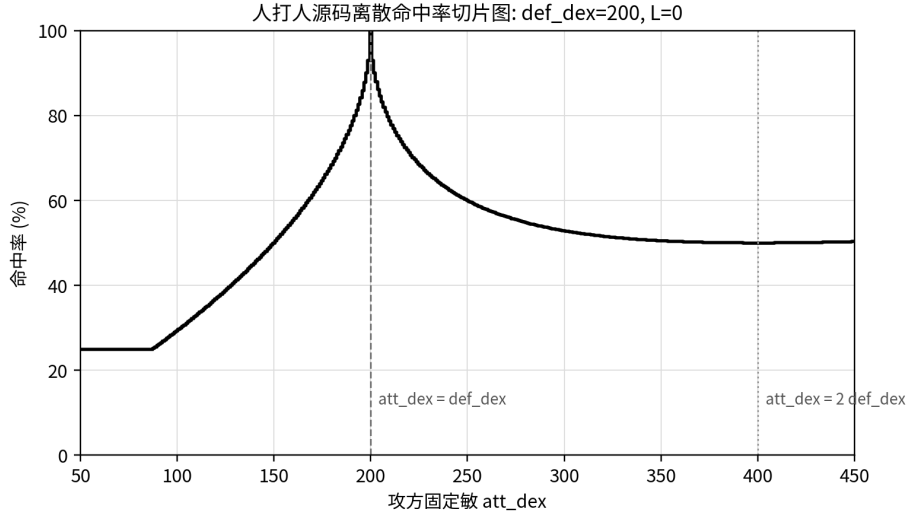


图 9.1: 人打人普通回避率热力图 ($L = 0$)

图 9.2: 人打人命中率切片图 ($def_{dex} = 200, L = 0$)

9.2.5 不宜忽略的实现次序

上述流程中有四个实现细节尤其需要保留。其一，身份修正按对象组合仅修正一侧敏捷。其二，弓修正在普通回避流程中累加两次。其三，AddHit 作用于 $[1, 7500]$ 裁切之后，而职业技能“回避”的倍率又作用于 AddHit 之后。其四，混乱攻击在上述步骤之后再将其回避值放大 40%。这些步骤的相对次序不能交换，否则会产生与源码不一致的边界结果。

9.3 会心率、武器字段与真实概率

9.3.1 会心公式的身份修正

会心判定使用独立于回避的身份修正规则。设攻击方与防守方原始固定敏仍为 att_{dex} 与 def_{dex} ，武器会心值为 C ，攻击方幸运为 L_a 。在当前主分支下，会心公式的身份修正可整理为

$$\widehat{(att_{dex}, def_{dex}, \beta, \rho)} = \begin{cases} (att_{dex}, \lfloor 0.8 def_{dex} \rfloor, 0.09, \sqrt{\cdot}), & \text{宠物打敌人,} \\ (att_{dex}, def_{dex}, 10, id), & \text{敌人打宠物,} \\ (att_{dex}, def_{dex}, 10, id), & \text{非玩家打玩家,} \\ (att_{dex}, \lfloor 0.6 def_{dex} \rfloor, 0.09, \sqrt{\cdot}), & \text{玩家打非玩家,} \\ (att_{dex}, def_{dex}, 0.09, \sqrt{\cdot}), & \text{其余情形.} \end{cases} \quad (9.20)$$

其中 ρ 记号用来区分“平方根口径”和“线性口径”：若 $\rho = \sqrt{\cdot}$ ，源码取平方根；若 $\rho = id$ ，则直接取一次差值，不再开根。

令

$$B = \max\{\widehat{att_{\text{dex}}}, \widehat{def_{\text{dex}}}\}, \quad S = \min\{\widehat{att_{\text{dex}}}, \widehat{def_{\text{dex}}}\}, \quad (9.21)$$

并定义

$$w = \begin{cases} 1, & \widehat{att_{\text{dex}}} \geq \widehat{def_{\text{dex}}}, \\ \frac{\widehat{att_{\text{dex}}}}{\widehat{def_{\text{dex}}}}, & \widehat{att_{\text{dex}}} < \widehat{def_{\text{dex}}}. \end{cases} \quad (9.22)$$

则会心基础体为

$$c_{\text{core}} = \begin{cases} \sqrt{\max\left\{\frac{B-S}{\beta}, 0\right\}} + 0.5C, & \rho = \sqrt{\cdot}, \\ \max\left\{\frac{B-S}{\beta}, 0\right\} + 0.5C, & \rho = \text{id}, \end{cases} \quad (9.23)$$

$$c_{\text{raw}} = c_{\text{core}} w + L_a. \quad (9.24)$$

源码随后执行

$$c_{\text{work}} = \min\{10000, \max\{1, \lfloor 100c_{\text{raw}} \rfloor\}\}, \quad (9.25)$$

并以 $\text{RAND}(1, 10000) < c_{\text{work}}$ 判定是否会心成立。

9.3.2 玩家打玩家的离散化简式

对玩家 PVP 而言，上述身份修正退化为 $\widehat{att_{\text{dex}}} = att_{\text{dex}}$ 、 $\widehat{def_{\text{dex}}} = def_{\text{dex}}$ 、 $\beta = 0.09$ 且采用平方根口径，于是连续基础体可写为

$$c_{\text{base}}(att_{\text{dex}}, def_{\text{dex}}, L, C) = \begin{cases} \sqrt{\frac{att_{\text{dex}} - def_{\text{dex}}}{0.09}} + L + 0.5C, & att_{\text{dex}} \geq def_{\text{dex}}, \\ \left(\sqrt{\frac{def_{\text{dex}} - att_{\text{dex}}}{0.09}} + 0.5C\right) \frac{att_{\text{dex}}}{def_{\text{dex}}} + L, & att_{\text{dex}} < def_{\text{dex}}. \end{cases} \quad (9.26)$$

对应的源码离散化会心率为

$$p_{\text{crit}}^{\text{src}}(att_{\text{dex}}, def_{\text{dex}}, L, C) = 0.01 \min\{9999, \max\{0, \lfloor 100c_{\text{base}}(att_{\text{dex}}, def_{\text{dex}}, L, C) \rfloor - 1\}\}. \quad (9.27)$$

之所以是减去 1，是因为源码最终使用的是严格小于号 $<$ ，而不是回避公式中的小于等于号 $\leq$ 。

装备说明中的“会心 +X”不能直接理解为“真实会心率增加 X%”。例如，若取最简口径 $att_{\text{dex}} = def_{\text{dex}}$ 且 $L = 0$ ，则换上一件 $\text{ITEM_CRITICAL}=2$ 的武器后，真实会心率约为 0.99%；若换上一件 $\text{ITEM_CRITICAL}=4$ 的武器，真实会心率约为 1.99%。因此，在未触发上下限裁切时，武器会心字段的真实收益大致相当于“每 2 点字段值增加约 0.99 个百分点”，而不是说明文本所暗示的一比一百分比对应。

弓在概率层面仍可触发会心事件，但源码在伤害层跳过了额外的会心伤害函数，因此弓的会心事件在伤害收益上被显著削弱。

9.4 反击的触发条件与离散公式

9.4.1 触发条件

反击是一个在多项前置条件全部满足后才进入概率判定的后验事件。首先，当前出手动作必须属于可反击类型，当前主分支主要对应普通攻击与无防御攻击。其次，反击方和被反击方都必须存活。再次，带有 CHAR_BATTLEFLG_ABIO 标记的单位不能触发反击⁸。最后，若任意一边使用弓、回力或投掷类远程武器，反击检查直接失败。

源码主循环还显式限制了反击链的最大往返次数，使其成为带有深度上限的离散链式事件。

9.4.2 玩家打玩家时的反击离散化简式

设反击方与被反击方固定敏分别为 att_{dex} 与 def_{dex} 。反击基础敏捷体与会心类似，但不带武器会心项。对玩家 PVP 简化口径，有

$$q_{base}(att_{dex}, def_{dex}) = \begin{cases} \sqrt{\frac{att_{dex} - def_{dex}}{0.08}}, & att_{dex} \geq def_{dex}, \\ \sqrt{\frac{def_{dex} - att_{dex}}{0.08}} \frac{att_{dex}}{def_{dex}}, & att_{dex} < def_{dex}. \end{cases} \quad (9.28)$$

记武器对武器系数为 F ，反击方幸运为 L 。若忽略 _SUIT_ADDENDUM 下的额外反击修正，则源码中的玩家反击工作值可写成

$$r_{counter} = 0.1F q_{base}(att_{dex}, def_{dex}) + L. \quad (9.29)$$

源码随后执行

$$r_{work} = 100r_{counter}, \quad (9.30)$$

并在 $r_{work} \leq 0$ 时将其设为 1。最终以

$$\text{RAND}(1, 10000) < r_{work} \quad (9.31)$$

判定反击是否成立。注意此处 r_{work} 在源码中是浮点类型，不存在上取整或减 1 的操作；比较运算符为严格小于 $<$ 。若讨论宠物反击，则还要额外加入 BATTLE_COM_S_NOGUARD 的附加值，并注意宠物分支会在进入随机判定前显式裁切到 100%。

格挡类装备的说明文本也不能直接按真实概率理解。以白石盾牌为例，其说明写作“格挡率 +2%”，但装备事件参数对应 COUNTER:20。若把其余项压到 0，则单靠这件盾牌就会提供约 19.99 个百分点的真实反击率，而不是 2 个百分点。由此可见，格挡类装备的说明文字与源码中真正进入反击公式的字段值并不属于同一量纲。

⁸注意 CHAR_BATTLEFLG_ABIO (值 $1 < < 6$) 与本回合忠诚失控标记 CHAR_BATTLEFLG_AIBAD (值 $1 < < 1$) 是不同的标志位；反击检查读取的是前者而非后者。

9.5 连续化分析与主导趋势

以下讨论只针对“人打人”这条玩家 PVP 支路的连续化主体，即暂时忽略回避、会心、反击公式中的取整、上下限裁切，以及严格小于或小于等于带来的 0.01% 级偏移。这样做的目的不是替代源码，而是把源码离散结果背后的主导趋势提取出来。

9.5.1 命中对攻方敏捷并非全局单调

先把式 (9.18) 与式 (9.19) 中的取整与裁切去掉，只保留连续主体。此时命中率对攻方敏捷的导数满足：当 $att_{dex} < def_{dex}$ 时，

$$\frac{\partial p_{hit}}{\partial att_{dex}} = \frac{1}{2\sqrt{0.02}\sqrt{def_{dex} - att_{dex}}} > 0, \quad (9.32)$$

因此攻方敏捷低于守方时，继续补敏一定提高命中。然而当 $att_{dex} > def_{dex}$ 时，

$$\frac{\partial p_{hit}}{\partial att_{dex}} = \frac{def_{dex}(att_{dex} - 2def_{dex})}{2\sqrt{0.02}att_{dex}^2\sqrt{att_{dex} - def_{dex}}}. \quad (9.33)$$

式 (9.33) 的符号取决于 $att_{dex} - 2def_{dex}$ ：当 $def_{dex} < att_{dex} < 2def_{dex}$ 时导数为负， $att_{dex} = 2def_{dex}$ 时导数为零， $att_{dex} > 2def_{dex}$ 时导数转正。因此，在 $(def_{dex}, 2def_{dex})$ 区间内继续增加攻方敏捷反而会降低命中率。

9.5.2 会心存在局部非单调区间

对式 (9.26) 的连续主体，若先固定 L 与 C ，当 $att_{dex} > def_{dex}$ 时有

$$\frac{\partial p_{crit}}{\partial att_{dex}} = \frac{1}{2\sqrt{0.09}\sqrt{att_{dex} - def_{dex}}} > 0, \quad (9.34)$$

因此攻方敏捷领先后，会心率单调上升。然而在 $att_{dex} < def_{dex}$ 且先忽略武器会心时，

$$\frac{\partial p_{crit}}{\partial att_{dex}} = \frac{2def_{dex} - 3att_{dex}}{2def_{dex}\sqrt{0.09}\sqrt{def_{dex} - att_{dex}}}. \quad (9.35)$$

由式 (9.35) 可知，当 $att_{dex} < 2def_{dex}/3$ 时，会心率随攻方敏捷增加而上升；当 $2def_{dex}/3 < att_{dex} < def_{dex}$ 时，会心率反而下降。武器会心 C 的存在会把这一拐点向右推移，但不能消除非单调结构本身。

9.5.3 反击的主导单调性与会心同型

对式 (9.31)，若暂时去掉上取整与边界裁切，只保留连续主体 $0.1Fq_{base} + L$ ，则当 $att_{dex} > def_{dex}$ 时有

$$\frac{\partial p_{counter}}{\partial att_{dex}} = \frac{F}{20\sqrt{0.08}\sqrt{att_{dex} - def_{dex}}} > 0. \quad (9.36)$$

而在 $att_{dex} < def_{dex}$ 时,

$$\frac{\partial p_{counter}}{\partial att_{dex}} = \frac{F(2def_{dex} - 3att_{dex})}{20def_{dex}\sqrt{0.08}\sqrt{def_{dex} - att_{dex}}}. \quad (9.37)$$

因此, 对固定正武器系数 F 而言, 反击在单调性上与“武器会心取 0 的会心公式”完全同型: 当 $att_{dex} < 2def_{dex}/3$ 时先升, 进入 $2def_{dex}/3 < att_{dex} < def_{dex}$ 后转为下降, 越过 $att_{dex} = def_{dex}$ 后重新单调上升。武器系数 F 的作用主要是对整条曲线做比例缩放, 而不是改变拐点位置。

9.5.4 变量位置不同, 收益性质就不同

综合三条链可以看到, 各修正变量进入公式的层次并不相同: 幸运大多表现为平移项; 武器会心仅作用于会心; 武器对武器系数仅作用于玩家反击; AddHit 作用于回避上限裁切之后; 职业技能“回避”又在 AddHit 之后做倍率放大。因此, 属性的边际收益需要回到其在判定链中的具体位置来评估。

9.6 离散数值图示与结果对照

图 9.3 与图 9.4 都只绘制“人打人”这条支路。具体地, 固定守方敏捷为 $def_{dex} = 200$, 并统一取守方幸运 $L_d = 0$ 、攻方幸运 $L_a = 0$ 、武器会心值 $C = 0$ 、玩家反击武器系数 $F = 10$, 同时忽略弓、醉酒、AddHit、职业技能“回避”、无防御攻击附加与混乱攻击修正。表中给出的命中率、会心率与反击率, 均按本章的源码离散化简式直接计算, 而不是用连续近似值代替。

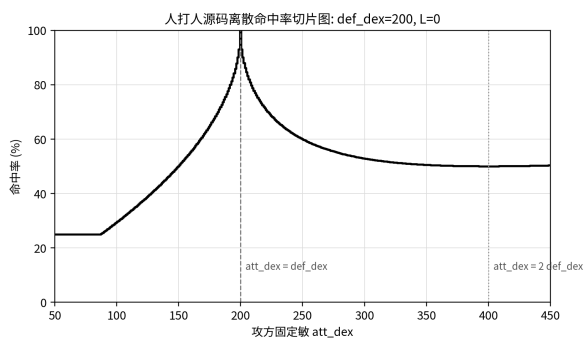
人打人源码离散概率表: $def_{dex}=200, L=0, C=0, F=10$

攻方 att_{dex}	命中率	会心率	反击率(F=10)	命中导数	会心导数	反击导数
80	25.00	14.59	15.49	升	升	升
120	36.76	17.87	18.97	升	升	升
133	42.13	18.13	19.24	升	升	升
160	55.28	16.85	17.88	升	降	降
200	99.99	0.00	0.00	降	升	升
240	62.74	21.07	22.36	降	升	升
280	54.83	29.80	31.62	降	升	升
320	51.59	36.50	38.72	降	升	升
400	50.00	47.13	49.99	平	升	升

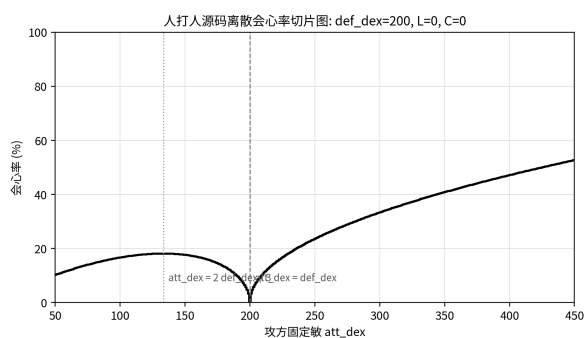
符号列使用连续化主体的一阶导号; 数值列按源码离散口径直接计算。

图 9.3: 固定守方敏捷 $def_{dex} = 200$ 时的源码离散概率表。统一取 $L_d = L_a = 0$ 、 $C = 0$ 、 $F = 10$, 并关闭弓、醉酒、AddHit、职业技能“回避”、无防御攻击附加与混乱攻击修正。表中符号列使用连续化主体的一阶导号, 只用于展示主导趋势。

图 9.4 进一步给出同一组参数下的切片图。三张图中的曲线都来自源码离散化简式, 因此会呈现出台阶状; 而虚线和点线标记的则是连续化主体中的关键阈值, 用于说明“源码离散结果”和“连续趋势分析”之间是如何对应的。



(a) 命中率切片图



(b) 会心率切片图

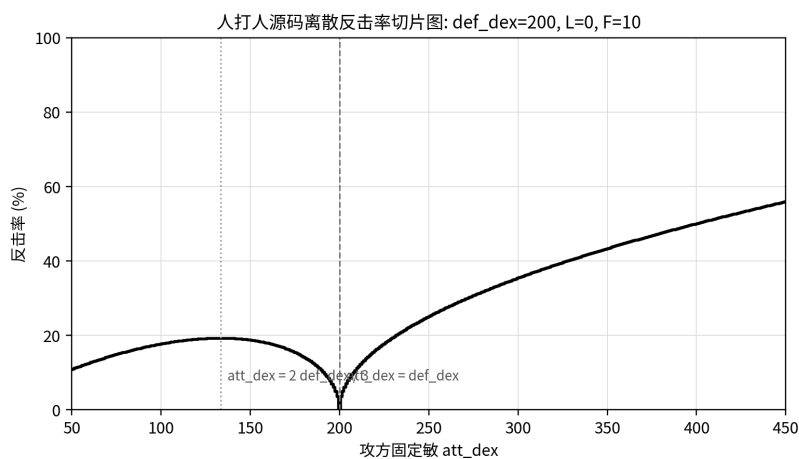
(c) 反击率切片图 ($F = 10$)

图 9.4: 玩家 PVP 简化口径下的源码离散切片图。统一取守方固定敏 $def_{dex} = 200$ 、幸运为 0；会心图额外取武器会心值 $C = 0$ ，反击图额外取武器系数 $F = 10$ 。图中的阈值线来自连续化主体，用于标记导数符号发生变化的位置。

图中可见，命中率在 $def_{dex} < att_{dex} < 2def_{dex}$ 区间出现局部下降，会心率与反击率在 $2def_{dex}/3 < att_{dex} < def_{dex}$ 区间出现局部下降。这表明，离散源码结果与前述连续化导数分析在主导趋势上是一致的。

第十章 法术、精灵与职业技能的命中和伤害分层

与物理判定主要围绕固定敏和固定幸运展开不同，法术系统改用原始幸运、属性抗性、抗魔装备和技能熟练度。更重要的是，法术机制在源码中天然分成“是否命中”与“命中后造成多少伤害”两层；若忽略这一分层，就很容易把抗性、熟练度与装备的作用位置混写。本章据此分别整理普通法术、精灵伤害和职业法术的命中结构，并说明命中层与伤害层之间的关系。

10.1 普通法术命中的对象分支

普通属性法术与精灵伤害的命中入口可统一理解为 `BATTLE_MagicDodge()`。若目标是玩家，则未命中基数写成

$$m_{\text{normal}} = \lfloor 3L + 0.15R + 0.9E \rfloor, \quad (10.1)$$

其中 L 是目标原始幸运 `CHAR_LUCK`， R 是目标对应属性抗性， E 是抗魔装备值。法术命中率于是为

$$p_{\text{magic}}^{(\text{player})} = 100 - m_{\text{normal}}. \quad (10.2)$$

若目标不是玩家，则系统不再读取幸运、抗性和抗魔装备，而改用等级近似：

$$m_{\text{normal}}^{(\text{npc})} = \min \{ \lfloor 0.2L_v \rfloor, 30 \}, \quad (10.3)$$

从而

$$p_{\text{magic}}^{(\text{npc})} = 100 - m_{\text{normal}}^{(\text{npc})}. \quad (10.4)$$

两式表明，普通法术对玩家目标与非玩家目标采用完全不同的输入变量集：前者依赖幸运、抗性与抗魔装备，后者退化为等级近似。此外，普通法术命中完全不读取生命值——当前生命、最大生命、生命成长档均不进入 `BATTLE_MagicDodge()`。生命只在后续伤害或以生命为基准的特殊技能中发挥作用。

需要补充说明的是，上述“非玩家目标按等级近似”只成立于命中层。当前工程里 `_FIX_MAGICDAMAGE` 处于开启状态，因此宠物目标在普通属性法术/精灵伤害的伤害层仍会读取自身的四系原始抗性 `CHAR_EARTH_RESIST` 到 `CHAR_WIND_RESIST`；但这些原始抗

性并不进入 `BATTLE_MagicDodge()`，所以它们会影响宠物承受的法术伤害强弱，却不会改变普通法术是否打中宠物这一层结果。

10.2 精灵与普通法术的同构命中链

精灵伤害与普通属性法术共用同一命中入口，适用式 (10.1)–式 (10.2) 的同一口径：玩家目标的命中主要由原始幸运、对应抗性和抗魔装备决定，非玩家目标退化为等级近似。

由于进攻方缺乏类似物理系的可堆叠命中主属性，精灵命中的优化空间主要集中在守方的抗性、抗魔装备和幸运配置上。

10.3 职业法术命中与熟练度人口

职业法术命中由 `PROFESSION_MAGIC_DODGE()` 负责，相比普通法术新增了攻方对应属性熟练度作为核心变量。若目标是玩家，则第一段未命中基数可整理为

$$m_{\text{prof}} = \lfloor 3L + 0.5R + 0.4E - 0.2P \rfloor, \quad (10.5)$$

其中 L 为目标原始幸运， R 为目标对应职业属性抗性， E 为抗魔装备， P 为攻方对应属性熟练度。于是第一段命中率为

$$p_{\text{prof},1} = 100 - m_{\text{prof}}. \quad (10.6)$$

若目标不是玩家，则幸运、装备项退化为等级近似，得到

$$m_{\text{prof}}^{(\text{npc})} = \lfloor \min\{0.15L_v, 20\} - 0.2P \rfloor. \quad (10.7)$$

因此，职业法术相比普通法术多了一个清晰可控的进攻端变量 P 。从边际结构看，玩家目标每增加 1 点原始幸运会让第一段命中率约下降 3%，而攻方每增加 5 点熟练度会让第一段命中率约提高 1%。这也是职业法术研究中“熟练度比幸运更像可优化变量”的根本原因。与普通法术一样，职业法术命中同样不读取任何生命值字段。

此外存在一个绝对前置条件：若目标当前处于地球一周，则职业法术直接 Miss，不进入式 (10.5) 的概率计算。该防御动作直接切断命中流程，而非降低命中率。

把两类法术命中并列起来，可以得到表 10.1。

表 10.1: 普通法术与职业法术命中的输入变量对照

项目	普通属性法术 / 精灵	职业法术
命中入口	BATTLE_MagicDodge()	PROFESSION_MAGIC_DODGE()
玩家目标主要输入	原始幸运、对应属性抗性、抗魔装备	原始幸运、对应职业属性抗性、抗魔装备、攻方熟练度
非玩家目标主要输入	目标等级近似, 最高压到 30	目标等级近似, 最高压到 20, 再减熟练度项
是否读取生命值	不读取	不读取
额外前置绝对 Miss	无这一步	若目标当前动作是 BATTLE_COM_S_EARTHROUND0, 直接 Miss
二段命中	无	部分技能存在固定二段命中

10.4 二段命中与状态技能的后续写入

职业技能里并不是所有法术在通过第一段命中后就直接生效。若把式 (10.6) 的第一段命中率记为 $p_{\text{prof},1}$, 则部分技能还会叠加一个固定第二段命中率 p_2 , 于是最终命中率写成

$$p_{\text{final}} = p_{\text{prof},1} p_2. \quad (10.8)$$

当前已核对的典型二段判定包括¹:

$$p_2 = \begin{cases} 74\%, & \text{电流术、暴风雨 (RAND(1,100)<75),} \\ 89\%, & \text{火龙枪、世界末日 (RAND(1,100)<90),} \\ 49\%, & \text{一针见血 (RAND(1,100)<50).} \end{cases} \quad (10.9)$$

冰爆术则是一个特殊例子。源码在原本的命中分支之后又直接把 hit 重写成 100, 因此就当前代码口径而言, 这一段额外命中应视为必定通过。这个例子说明, 研究法术命中不能仅依据技能说明文本, 而必须回到实际实现顺序, 因为后续覆写足以改变整段逻辑。

对状态技能而言, 还需进一步区分“命中成功”和“状态参数如何写入”。例如四属性结界会在命中后写入结界槽位; 挑拨、树根缠绕等则会继续改写攻防敏工作值。换言之, 命中本身只是状态技能的入口, 而不是其最终效果。

¹源码使用 $\text{RAND}(1,100)<N$ 判定 (闭区间整数、严格小于), 因此实际通过率分别为 74/100、89/100 和 49/100, 比直觉上的 75%、90% 和 50% 各低 1%。

10.5 命中层与伤害层的分离

法术是否命中，与命中后造成多少伤害，是两层彼此独立的计算。若把法术基础威力记为 M ，攻方对应熟练度记为 P ，守方对应抗性、抗装、精灵抗性分别记为 R, S, G ，则项目当前整理出的职业法术伤害层可写为

$$damage = M \left(1 + \frac{P}{100}\right) \left(1 - \frac{R}{100}\right) \left(1 - \frac{S}{100}\right) \left(1 - \frac{G}{100}\right). \quad (10.10)$$

式 (10.10) 与前面的命中公式共同说明：提高法术表现有两条完全不同的路径。第一条是让 p_{final} 上升，也就是更容易打中；第二条是让式 (10.10) 上升，也就是打中了更疼。熟练度同时参与这两层，因此往往是职业法术里最具复合收益的进攻变量；而抗性则既能降低第一段命中，又能在命中后继续减伤，因此同样具有双重防守价值。

命中层与伤害层的分离使法术优化成为多变量问题：稳定控场优先提高 p_{final} ，单次爆发则侧重式 (10.10) 的最大化。

还要补一个容易混淆的对象差异：式 (10.10) 中的 G 不是“宠物自带精灵抗性”，而是由对应抗性状态存在时读取的 `CHAR_OTHERSTATUSNUMS`。当前源码里，宠物生成流程没有发现会在出生阶段主动写入非零四系原始抗性或负抗性的路径；默认角色模板四项初值为 0，敌人/宠物生成时也未额外赋值。因此，若讨论“宠物自带抗性”，更准确的说法应是：宠物持有并持久化四系原始抗性字段，但其初始值默认是 0；这些值在当前编译口径下能参与普通属性法术/精灵的伤害层，并可在相关法术战斗经验流程中继续增减，而不是一个独立的出生负抗系统。

第十一章 骑宠混合与物理伤害结构

骑宠状态下，人物与宠物的攻击、防御与敏捷并不是简单叠加，而是按武器类型、攻守位置和后续公式入口分别混合。混合后的结果又继续进入物理伤害、属性修正与会心附加伤害。因此，骑宠不仅改变了多个核心变量的数值大小，也改变了这些变量进入伤害公式的层级与路径。本章的目标是把这种“先混合、后分层结算”的结构明确写出。

11.1 骑宠攻防混合的基本结果

骑宠三围的读取主要对应 `BATTLE_adjustRidePet3A()`，并以 `_BATTLE_NEWPOWER` 分支为主。由于远程与近战的混合方式不同，以下分别讨论。

设人物攻击力、防御力分别为 A_c, D_c ，骑宠攻击力、防御力分别为 A_p, D_p 。若攻击方未骑宠，则物理攻击直接取

$$A = A_c. \quad (11.1)$$

若攻击方已骑宠，则当装备远程武器时，攻击力混合为

$$A = A_c + 0.4A_p, \quad (11.2)$$

而当装备近战武器或空手时，攻击力混合为

$$A = 0.8A_c + 0.8A_p. \quad (11.3)$$

骑宠近战时人物与宠物攻击共同占据较高权重（各 0.8），骑宠远程则偏向人物本体，宠物仅作为 0.4 的附加项。

防御侧的结构更特殊。若防守方未骑宠，则在 `BATTLE_DamageCalc()` 中会先取

$$D = 0.70D_c. \quad (11.4)$$

若防守方已骑宠，则首先在骑宠混合函数中得到

$$D_{ride} = 0.7D_c + 0.7D_p, \quad (11.5)$$

随后再进入伤害函数乘以一次 0.70，于是最终用于物伤计算的防御量为

$$D = 0.70D_{ride} = 0.49D_c + 0.49D_p. \quad (11.6)$$

因此，在当前编译口径下，骑宠防御既不是人物防御与宠物防御的简单平均，也不是只取宠物，而是对两者做同权线性组合后再统一衰减一次。

11.2 骑宠敏捷混合与攻守差异

骑宠敏捷的处理与攻击、防御不同，它既要区分“攻击侧读取”与“防守侧读取”，也要区分远程武器与近战武器。设人物战斗速度敏为 Q_c ，骑宠战斗速度敏为 Q_p 。则在攻击侧，若使用远程武器，读取为

$$Q_{atk}^{(ranged)} = 0.8Q_c + 0.2Q_p, \quad (11.7)$$

若使用近战武器或空手，则读取为

$$Q_{atk}^{(melee)} = 0.2Q_c + 0.8Q_p. \quad (11.8)$$

而在防守侧，读取进一步偏向宠物，写成

$$Q_{def} = 0.1Q_c + 0.9Q_p. \quad (11.9)$$

骑宠近战敏捷偏向宠物 ($0.8Q_p$)，骑宠远程偏向人物 ($0.8Q_c$)，骑宠防守则几乎由宠物敏捷主导 ($0.9Q_p$)。同一只宠物在“骑近战”和“骑远程”两种配置下，可能表现出完全不同的先手性质，也会在伤害、命中与反击中体现不同的角色定位。

11.3 基础物伤的三段结构

设前两节得到的攻击量与防御量分别记为 A 与 D 。在当前 `_BATTLE_NEWPOWER` 分支下，物理伤害的基础体可分成三段。若

$$D > A, \quad (11.10)$$

则基础物伤只在极低范围内波动，可整理为

$$damage = \text{RAND}(0, 1). \quad (11.11)$$

若

$$D \leq A < \frac{8}{7}D, \quad (11.12)$$

则进入第二段：

$$damage = \text{RAND}\left(0, \frac{A}{16}\right). \quad (11.13)$$

若

$$A \geq \frac{8}{7}D, \quad (11.14)$$

则进入第三段，记

$$K_0 = \text{RAND}\left(0, \frac{A}{8}\right) - \frac{A}{16}, \quad (11.15)$$

则基础物伤为

$$damage = 2(A - D) + K_0. \quad (11.16)$$

其中当前源码常量对应

$$DAMAGE_RATE = 2.0, \quad D_8 = \frac{1}{8}, \quad D_16 = \frac{1}{16}. \quad (11.17)$$

当攻击显著低于防御时，伤害几乎被压到 0 或 1；当攻击刚刚接近防御时，仍停留在小幅随机区间；只有当 $A \geq 8D/7$ 之后，伤害才表现为对 $(A - D)$ 的线性函数并叠加轻微随机扰动。

11.4 防御动作与后置减伤链

当前物理伤害流程里还存在一条独立的“防御链”。它不直接改写 CHAR_WORKDEFEN
CEPOWER，而是在基础伤害已经算出之后，再按动作状态或专门判定去压低最终伤害。

其中最基础的一层是防御动作。若防守方当前动作是 BATTLE_COM_GUARD，且未处于混乱，则源码不会简单按固定比例减伤，而是调用 BATTLE_GuardAdjust() 做一次六档随机压缩：

$$damage_{guard} \in \{0, 0.1, 0.2, 0.3, 0.4, 0.5\} \cdot damage, \quad (11.18)$$

其概率分布分别为

$$(25\%, 25\%, 20\%, 15\%, 10\%, 5\%). \quad (11.19)$$

防御动作以完全挡下到保留一半伤害之间的离散随机分布实现。若压缩后伤害降到 0，系统会把这次结果标记为 BATTLE_RET_ALLGUARD，在表现层显示为防御成功。

第二层是偏转 / 招架。若编译路径启用了 _EQUIT_ARRANGE，则在普通防御动作之外，还会额外调用 BATTLE_ArrangeCheck()。一旦成立，本次伤害会被直接压到原值的 10%，并返回独立的偏转结果码。这一层本质上是“招架成功后大幅卸力”，独立于普通防御动作。

第三层是忽视防御。若编译路径启用了 _EQUIT_NEGLECTGUARD，且攻击方装备或状态给出了 CHAR_WORKNEGLECTGUARD，则它并不是去跳过“防御动作”，而是在基础物伤计算前先按比例削减防守方防御量。若忽视防御比例记为 $r\%$ ，则进入物伤公式的防御量会先变成

$$D' = \left(1 - \frac{r}{100}\right) D. \quad (11.20)$$

因此，忽视防御影响的是三段式物伤的分段位置，而不是后面的防御动作随机压缩。

把这几层放到同一条顺序里，可以更准确地写成：先由攻击力与防御力工作值得到基础物伤，再经过属性修正与结界减伤，然后再检查防御动作、偏转和伤害反应等后置环节。于是，玩家口中的“被格挡了”在源码里其实可能对应三种完全不同的机制：普通防御动作的随机减伤、偏转 / 招架的 10% 压缩，或吸收 / 反射 / 消失等伤害反应。它们在日志表现上都可能让单次伤害显著下降，但进入公式的位置并不相同。

11.5 属性层、场地与结界减伤

基础物伤只是整个伤害流程的第一层。对当前实现而言，式 (11.11)–(11.16) 得到的结果还要继续经过属性克制、场地倍率与结界减伤，然后才进入额外扰动与后续表现层。这一部分由五维属性向量、双线性相克矩阵与场地比值共同组成。

11.5.1 攻击方与防守方的五维属性向量

在当前 `_BATTLE_NEWPOWER` 口径下，物理伤害使用的属性值由 `BATTLE_GetAttr()` 直接从单位自身的四属性工作值读取。骑宠会混合攻击、防御与敏捷，但不会混合物理属性向量。若把攻击方四属性写成

$$(p_E, p_W, p_F, p_{Wi}), \quad (11.21)$$

把防守方四属性写成

$$(q_E, q_W, q_F, q_{Wi}), \quad (11.22)$$

则源码会先把负值裁到 0，再补出无属性分量

$$p_N = \max\{100 - p_E - p_W - p_F - p_{Wi}, 0\}, \quad q_N = \max\{100 - q_E - q_W - q_F - q_{Wi}, 0\}. \quad (11.23)$$

于是攻守双方真正进入属性克制计算的，分别是五维向量

$$\mathbf{p} = (p_E, p_W, p_F, p_{Wi}, p_N), \quad \mathbf{q} = (q_E, q_W, q_F, q_{Wi}, q_N). \quad (11.24)$$

式 (11.23) 说明，只要四属性之和没有达到 100，剩余部分就会自动落入无属性；若四属性和已经超过 100，则无属性被截成 0，而不会出现负的属性值。

11.5.2 属性克制的源码矩阵

记式 (11.11)–(11.16) 的基础物伤为 $damage_0$ 。源码会先把攻击方五维属性按比例拆到这份基础物伤上，再与防守方五维属性逐项相乘，并乘上三种常量

$$AJ_{\text{up}} = 1.5, \quad AJ_{\text{same}} = 1.0, \quad AJ_{\text{down}} = 0.6. \quad (11.25)$$

若按 (E, W, F, Wi, N) 的顺序排列攻守属性，则源码等价于使用如下相克矩阵

$$M = \begin{bmatrix} 1.0 & 1.5 & 1.0 & 0.6 & 1.5 \\ 0.6 & 1.0 & 1.5 & 1.0 & 1.5 \\ 1.0 & 0.6 & 1.0 & 1.5 & 1.5 \\ 1.5 & 1.0 & 0.6 & 1.0 & 1.5 \\ 0.6 & 0.6 & 0.6 & 0.6 & 1.0 \end{bmatrix}. \quad (11.26)$$

于是属性克制后的物伤主体可以直接写成

$$damage_{attr} = damage_0 \frac{\mathbf{p}^T M \mathbf{q}}{10000}. \quad (11.27)$$

式 (11.26) 有几个值得注意的性质。其一，相克链为土克水、水克火、火克风、风克土。其二，同属性与“隔一位”的属性关系在当前物伤克制里都按 1.0 处理。其三，无属性并非中立：元素打无属性按 1.5 算，无属性打任意元素按 0.6 算，只有无属性打无属性才是 1.0。

若只看纯属性对纯属性，则式 (11.27) 可以压缩成表 11.1。

表 11.1: 物理伤害中纯属性对纯属性的源码倍率

攻 \ 守	土	水	火	风	无
土	1.0	1.5	1.0	0.6	1.5
水	0.6	1.0	1.5	1.0	1.5
火	1.0	0.6	1.0	1.5	1.5
风	1.5	1.0	0.6	1.0	1.5
无	0.6	0.6	0.6	0.6	1.0

11.5.3 场地倍率不是加法，而是攻守比值

属性克制并不是这一层的终点。源码接着会读取战场属性 `field_att` 与场地强度 `att_pow`，分别对攻击方与防守方算一次场地倍率，然后取二者比值乘回伤害。设当前场地属性为 $X \in \{E, W, F, Wi\}$ ，场地强度为 s ，则对任意一方属性向量 $\mathbf{u}$ ，其场地倍率可写成

$$\phi_X(\mathbf{u}) = 0.5 + 0.5 u_X \frac{s}{10000}, \quad (11.28)$$

若当前没有属性场地，则源码回到默认值

$$\phi_{\text{none}}(\mathbf{u}) = 0.5. \quad (11.29)$$

于是属性与场地共同作用后的结果是

$$damage_1 = damage_{attr} \frac{\phi_X(\mathbf{p})}{\phi_X(\mathbf{q})}. \quad (11.30)$$

式 (11.30) 的一个直接推论是：场地不是简单给攻击方加成，也不是简单给防守方减伤，而是对攻守双方同时各算一次倍率再取比值。因此，同一个火场里，火高的攻击方确实会放大输出，但火高的防守方也会通过分母把这部分收益抵消掉一部分。

11.5.4 四属性结界的减伤公式

在 `_PROFESSION_ADDSKILL` 分支下，物伤经过式 (11.30) 后，还会再检查防守方是否挂有四属性结界。这里的实现有三个关键细节。

第一，结界检查是按

$$E \rightarrow W \rightarrow F \rightarrow Wi \quad (11.31)$$

的 `if-elseif` 顺序进行，因此同一时刻即使理论上挂了多个结界，也只会命中第一条成立的分支。第二，减伤读取的是攻击方原始固定属性工作值，而不是式 (11.30) 里已经参与过场地和相克计算的属性向量。第三，结界高位里存放的“强度值”在当前物伤减伤公式中并不直接进入比例，只起到“该结界是否开启”的开关作用。

若记命中的结界属性为 $k \in \{E, W, F, Wi\}$ ，记攻击方对应的固定属性工作值为 r_k ，则当前源码下的结界减伤可写成

$$damage_2 = damage_1 \left(1 - \frac{r_k}{200}\right). \quad (11.32)$$

若没有任何结界成立，则直接有

$$damage_2 = damage_1. \quad (11.33)$$

式 (11.32) 说明，结界并不是“固定减半”或“按结界强度减伤”，而是按攻击方同属性点数做线性压缩。例如攻击方火属性固定值为 40，而防守方当前触发的是火结界，则物伤会再乘上 $1 - 40/200 = 0.8$ 。

11.5.5 属性反转如何进入本章伤害公式

如第 十三章所述，属性反转通过标志位控制当前固定属性快照的交换。在伤害层面，它影响的是本节前面几条公式的输入量本身。

若某单位当前有效固定属性由式 (13.14) 从

$$(E, W, F, Wi) \mapsto (F, Wi, E, W) \quad (11.34)$$

交换而来，那么在当前主分支下至少有三处会随之改变。

第一，`BATTLE_GetAttr()` 读取到的攻击方或防守方属性向量 $\mathbf{p}, \mathbf{q}$ 会改成反转后的版本，因此式 (11.27) 中的双线性相克值会整体改变。第二，式 (11.28) 中用于计算场地倍率的对应属性分量 u_X 也会改成反转后的版本，因此同一块场地上的攻守比值可能随反转而跳变。第三，式 (11.32) 中的 r_k 直接读取攻击方当前固定属性工作值，所以结界减伤也会跟着反转后的属性一起变化。

因此，若把“反转前”的伤害记为

$$damage_2 = \mathcal{D}(\mathbf{p}, \mathbf{q}, r_k), \quad (11.35)$$

把“反转后”的伤害记为

$$damage_2^{(rev)} = \mathcal{D}(\mathbf{p}^{(rev)}, \mathbf{q}^{(rev)}, r_k^{(rev)}), \quad (11.36)$$

则差异不止发生在某一步，而是会同时穿过

$$\mathbf{p}^\top M \mathbf{q}, \quad \frac{\phi_X(\mathbf{p})}{\phi_X(\mathbf{q})}, \quad 1 - \frac{r_k}{200} \quad (11.37)$$

这三层结构。

一个重要的时序细节是：当前物伤函数只读取当前这份已经算好的固定属性工作值，不会重新检查反转标志。若出现第 13 章讨论过的过渡情形（反转标志已被清掉，但新的固定属性重算尚未发生），本章中的 $\mathbf{p}, \mathbf{q}, r_k$ 仍会继续使用尚未恢复的反转态快照。

11.5.6 两个完整算例

为了避免把式 (11.27)–(11.32) 看成抽象符号，这里给出两个按当前源码口径直接展开的例子。

例 1：属性克制与场地比值共同放大伤害 设基础物伤 $damage_0 = 120$ ，攻击方属性向量为 $\mathbf{p} = (60, 20, 10, 0, 10)$ ，防守方属性向量为 $\mathbf{q} = (10, 50, 20, 0, 20)$ ，当前为土场地，场地强度 $s = 80$ 。按式 (11.27) 与式 (11.30) 展开，可把关键中间量压缩成表 11.2。

表 11.2：算例 1：属性克制与土场地的联合放大

步骤	计算内容	结果
基础物伤	已知 $damage_0$	120
属性克制主体	$\mathbf{p}^\top M \mathbf{q}$	12000
属性克制后伤害	$120 \times 12000 / 10000$	144
攻方土场倍率	$0.5 + 0.5 \times 60 \times 80 / 10000$	0.74
守方土场倍率	$0.5 + 0.5 \times 10 \times 80 / 10000$	0.54
场地修正后伤害	$144 \times 0.74 / 0.54$	≈ 197.33

属性克制先把伤害从 120 推高到 144，土场地又因攻击方土属性显著高于防守方土属性，继续推到约 197.33。场地收益在属性克制已经放大的基础上继续按攻守比值放大。

例 2：属性反转同时改变场地收益与结界减伤 设基础物伤 $damage_0 = 100$ ，防守方为纯无属性 $\mathbf{q} = (0, 0, 0, 0, 100)$ ，当前为火场地、场地强度 $s = 100$ ，且防守方身上存在土结界。攻击方反转前的当前固定属性为 $\mathbf{p} = (80, 20, 10, 5, 0)$ ，按第 13 章的反转映射，反转后的当前固定属性变为 $\mathbf{p}^{(rev)} = (10, 5, 80, 20, 0)$ 。关键中间量汇总见表 11.3。

表 11.3: 算例 2: 属性反转如何同时改写场地与结界

步骤	反转前	反转后
攻方当前属性	(80, 20, 10, 5, 0)	(10, 5, 80, 20, 0)
属性克制后伤害 $damage_{attr}$	172.5	172.5
火场攻方倍率	0.55	0.9
火场守方倍率	0.5	0.5
场地修正后伤害 $damage_1$	189.75	310.5
土结界读取的攻 方土属性	80	10
结界修正后伤害 $damage_2$	113.85	294.975

这个例子非常典型：反转前后，属性克制主体本身没有变，但火场地倍率和土结界减伤同时被改写，最终伤害仍然从 113.85 大幅跳到约 294.98。因此，实战里看到“反转后伤害忽然暴涨”，并不一定意味着攻防差或会心出了问题，很多时候只是属性向量在场地与结界两个层面同时改了方向。

11.5.7 把属性层并入最终伤害顺序

把本章前文与本节合并后，当前主分支下的物理伤害顺序可以整理为

$$damage_0 \xrightarrow{\text{属性克制矩阵}} damage_{attr} \xrightarrow{\text{场地比值}} damage_1 \xrightarrow{\text{四属性结界}} damage_2 \xrightarrow{\text{额外扰动}} damage_3. \quad (11.38)$$

其中若编译启用了额外伤害与额外减伤系统，则附加扰动写成

$$\Delta_{extra} = \text{RAND}(0.3P_a, P_a) - \text{RAND}(0.3P_d, P_d), \quad (11.39)$$

并有

$$damage_3 = \max\{damage_2 + \Delta_{extra}, 0\}. \quad (11.40)$$

完整的物伤主体可以概括为“攻防分段函数 + 五维属性双线性克制 + 场地攻守比值 + 结界线性减伤 + 额外扰动”的串联结构。在骑宠环境下，攻击、防御跟随骑宠混合，但属性向量仍读取当前单位自身工作值，二者在源码中相互独立。

11.6 会心事件的伤害层附加

会心成立后，系统在普通物伤基础上增加一段由防守方防御与双方等级共同决定的附加伤害，而非乘以固定倍率。若武器不是弓，记普通物伤结果为 $damage$ ，攻击方等级与防守方等级分别为 L_a 与 L_d ，则会心附加后的结果可整理为

$$damage_{crit} = damage + 0.5 DEF_c \frac{L_a}{L_d}, \quad (11.41)$$

其中 DEF_c 表示防守方的防御力工作值。式 (11.41) 说明，会心本质上是“普通物伤 + 一段额外防御比例项”，而不是传统 RPG 中常见的“普通物伤乘一个暴击倍数”。

会心附加量与防守方防御及双方等级比直接相关，对不同目标的收益显著不同：防高、等级低的目标吃到更大的会心附加，防低、等级接近的目标增幅较小。弓虽然仍可判出会心事件，但不会调用这段附加伤害函数，因此弓的概率层面会心与爆发层面会心是两个独立问题。

11.7 变量收益的结构性差异

11.7.1 攻击对伤害的边际收益是分段的

由式 (11.11)–(11.16) 可见，攻击力提升的边际收益呈分段特征。当 $A < D$ 时，提高攻击仅使随机区间从“几乎固定为 0 或 1”逐步靠近第二段，边际收益极弱；当 $D \leq A < 8D/7$ 时，伤害上界随 $A/16$ 线性增加，但仍停留在小区间随机；只有跨过 $8D/7$ 阈值后，攻击提升才按 $2(A - D)$ 的斜率进入主导区间。长期处在第二段附近的配置，继续加攻击的实际收益远低于线性预期。

11.7.2 骑宠远程与骑宠近战的收益结构不同

由式 (11.2)、(11.3) 及式 (11.7)、(11.8) 可知，骑宠远程与骑宠近战的攻击和敏捷权重结构不同。远程侧攻击和敏捷均偏人物，近战侧则让人物和宠物以接近等权进入攻击项，敏捷偏向宠物。因此“提高宠物攻击”与“提高宠物敏捷”的收益与武器类型直接耦合。

11.7.3 防御与石化、忽视防御等状态的作用顺序

防御量在进入伤害三段式之前，还可能受到铁壁防御、石化、无装备防御和忽视防御等状态影响。这些修正发生在不同位置：石化直接把防御翻倍，整体抬高第三段门槛；忽视防御在防御量已经形成之后按比例削减，相当于对门槛的整体下压；无装备防御类技能则将防御替换为另一套工作值。各修正不仅改变数值大小，还可能改变分段函数所处区间。

骑宠与物伤系统共同构成一个高度离散、分段且带路径依赖的结构。攻击、防御、武器类型、骑宠状态、属性结界和会心按特定顺序进入不同层级的公式。前文宠物成长与喂药所产生的数值差异，最终通过这些分段函数与混合公式转化为实战中的先手、压制、爆发与承伤差异。

第十二章 装备字段、作用机制与案例研究

装备的战斗作用取决于字段的写入位置、工作值的承接方式以及后续判定链的读取路径，而不取决于说明文本本身。即使都写着“回避率上升”，不同装备也可能进入完全不同的实现路径；同样写着“增加攻击顺序”，其效果也未必等于直接提高 WORKQUICK。因此，本章不按装备部位或描述文字分型，而是把装备视为观察“字段入口如何决定机制解释”的案例研究对象，并以若干已核实装备说明字段量纲与真实效果之间的偏差。

12.1 装备字段的分类口径

表 12.1: 装备字段的主要作用机制总览

类型	字段 / 写入槽位	常见部位	当前可确认作用
直接属性型	ITEM_MODIFYATTACK/DEFENCE/QUICK	武器、防具、鞋	直接改当前回合攻、防、速工作值
资源上限型	ITEM_MODIFYHP/MP	铠、兜、服	直接改最大 HP / MP
概率基础型	ITEM_MODIFYLUCK, ITEM_MODIFYCHARM	饰品	分别进入固定幸运与固定魅力槽，二者口径不同
异常抗性型	ITEM_POISON 等	羽、饰品、套装	直接改传统异常抗性工作值
排序加项型	ITEM_MODIFYSEQUENCE	鞋	不改 WORKQUICK，而是在最终排序时给顺序值加项
会心专用型	ITEM_CRITICAL	武器	当前主会心公式直接读取武器槽 ITEM_CRITICAL
反击专用型	COUNTER	盾牌、少量武器 / 防具 / 套装	直接加进反击率公式
伤害尾段型	ITEM_OTHERDAMAGE/OTHERDEFC	武器、兜、铠	在最终伤害末段追加随机增伤 / 减伤项
谨慎项	ITEM_MODIFYAVOID	耳饰、项链、戒指、少量防具	当前只确认写入 CHAR_WORKFIXAVOID，未证实直接进入普通回避主体

上述分类的核心含义在于：装备应按字段入口分型，而不能按说明文本直接归类。部位只能说明“某类装备常见哪些字段”，真正决定战斗效果的仍然是字段写入点与后续读取点。

12.2 直接属性字段的工作值入口

装备效果表的基础映射可整理为

$$\begin{aligned} \text{ITEM_MODIFYATTACK} &\rightarrow \text{WORKATTACKPOWER}, \\ \text{ITEM_MODIFYDEFENCE} &\rightarrow \text{WORKDEFENCEPOWER}, \\ \text{ITEM_MODIFYQUICK} &\rightarrow \text{WORKQUICK}, \end{aligned} \quad (12.1)$$

$$\begin{aligned} \text{ITEM_MODIFYHP} &\rightarrow \text{WORKMAXHP}, \\ \text{ITEM_MODIFYMP} &\rightarrow \text{WORKMAXMP}, \end{aligned} \quad (12.2)$$

$$\begin{aligned} \text{ITEM_MODIFYLUCK} &\rightarrow \text{WORKFIXLUCK}, \\ \text{ITEM_MODIFYCHARM} &\rightarrow \text{WORKFIXCHARM}. \end{aligned} \quad (12.3)$$

上述字段在装备生效时直接写入当前回合工作值。其中，幸运与魅力虽常在玩家语境中被并称为“概率属性”，但在源码中的作用位置并不相同：固定幸运 WORKFIXLUCK 会进入普通回避、会心与反击等概率链；固定魅力 WORKFIXCHARM 在当前已核对的物理主体公式中并不进入命中、回避或会心判定。

“幸福的花圈”适合说明这一点：它同时带有“运 +2”与“魅 +20”，但前者进入固定幸运链，后者进入固定魅力链，二者不宜合并理解。“小耳环”“定情项链”则更适合作为纯魅力样例，而不宜直接写入概率公式样例。

12.3 传统异常抗性的适用边界

当前源码会把 ITEM_POISON、ITEM_PARALYSIS、ITEM_SLEEP、ITEM_STONE、ITEM_DRUNK、ITEM_CONFUSION 分别写入对应的传统异常抗性工作槽；后续传统异常状态命中链再统一读取这组 work 值。因此，这类装备更准确的表述应为“直接降低对应传统异常状态的命中概率”，而不是“追加一次独立抗性补判”。

对职业控制还需进一步区分。源码中“盾击”的眩晕和“天罗地网”的控制，走的是职业状态命中路径；当前已核对到的成功判定并不读取上述传统异常抗性槽，也未读取装备隐藏抗性里与 CHAR_WORKEQUITWEAKEN、CHAR_WORKEQUITBARRIER、CHAR_WORKEQUITNOCAST 等对应的装备抗性槽。就当前源码能够确认的路径而言，五素之羽的全异常抗性不会降低被盾击眩晕或被天罗地网控制的概率。与此同时，当前也未发现装备侧写入 CHAR_WORKMODDIZZY、CHAR_WORKMODDRAGNET 这类职业控制抗性槽的路径。

“五素之羽”是这一组最适合进入正文的样例。其“全异常抗性 +30”可以直接确认成立，因为这正对应上述传统异常抗性链；但同一件装备上的“回避率 +20%”并不能据此写成普通回避率固定加成。原始物品数据中，它对应的是 ITEM_MODIFYAVOID=10，而不是与说明文本一致的“20”；再加上当前尚未发现 CHAR_WORKFIXAVOID 进入普通回避主体的路径，因此这一部分仍应采用保守表述。

因此，五素之羽可以作为“传统异常抗性型”样例写入正文，但不宜作为“普通回避型”样例来概括。

12.4 行动顺序修正字段的独立入口

鞋子是最容易把两类速度收益混写的部位。其一是普通的“敏 +X”，它直接写入 WORKQUICK；其二是“增加攻击顺序 X%”，它走的是另一条独立链：

$$\begin{aligned} \text{ITEM_MODIFYSEQUENCE} &\rightarrow \text{CHAR_WORKFIXSEQUENCE} \\ &\rightarrow \text{CHAR_WORKSEQUENCEPOWER} \\ &\rightarrow \text{EntryList.sequence.} \end{aligned} \quad (12.4)$$

最终比较顺序时，源码读取的是

$$S_{\text{order}} = \text{DEX}_{\text{battle}} + \text{sequence}. \quad (12.5)$$

这意味着“增加攻击顺序 X%”并不改写 WORKQUICK，也不在 BATTLE_DexCalc() 内再次乘上倍率；它只是对最终排序值增加一个独立加项。

这一机制应与“急速攻击”严格区分。当前源码下，“急速攻击”是先取 WORKQUICK+20，再整体乘 1.3；而鞋子的顺序字段只是把一个独立的 `sequence` 加到最终排序值上。前者属于动作速度档，后者属于装备排序修正，二者并不处于同一层次。

“若丝鞋”和“白石鞋”是最适合说明这一点的样例。前者对应“敏 +5、增加攻击顺序 3%”，后者对应“敏 +8、增加攻击顺序 5%”。二者都同时携带 `quick` 与 `sequence` 字段，因此很适合说明：同样是“加快出手”，源码中至少存在“改 WORKQUICK”与“改最终排序加项”两条不同入口。

12.5 会心字段与武器槽量纲

当前主会心公式最关键的结论，是“直接读取武器槽道具的 ITEM_CRITICAL”，而不是读取全身累计后的某个会心工作值。源码先取攻击方武器槽 CHAR_ARM，再直接读取该道具的 ITEM_CRITICAL，随后并入主会心公式。

因此，当前主会心公式的装备入口应理解为武器槽专用字段。

这一界定有两个直接后果。第一，武器是讨论会心装备时最清晰的样例来源。第二，一些写着“会心一击率上升”的非武器描述，即便数据中也可能带有关联字段，只要尚未单独核清其读取路径，就不宜拿来代表主会心公式。

“小棍棒”和“普通普通的棍棒”是这一组最合适的样例。它们的价值不在于强度，而在于可以明确展示“会心字段来自武器槽”这一事实。若把人打人、攻守固定敏相等、双方幸运均为 0 作为最简口径，则武器会心值 C 的真实增益满足

$$p_{\text{crit}}^{\text{src}} \approx 0.5C - 0.01\%. \quad (12.6)$$

例如，若某件武器提供 `ITEM_CRITICAL=2`，则在该口径下真实会心率约为 0.99%，而不是 2%。这说明装备说明里的“会心 +X”并不能直接按“真实会心率 +X%”理解；它进入公式后的量纲要经过一次 0.5 缩放，并再经过离散比较带来的 0.01% 级偏移。

还要再区分会心概率层与会心伤害层。当前实现中的弓在概率层面仍可触发会心事件，但在伤害层会跳过通常的额外会心伤害收益。因此，本章只讨论“武器槽 `ITEM_CRITICAL` 如何进入会心概率公式”，而不把不同武器类型的会心伤害差异并入本节。

12.6 反击字段与“格挡率”文本偏差

盾牌上的“格挡率 +X%”常被误写成“额外防御”或“单独挡一次”。但当前源码中，`COUNTER` 字段会写入 `CHAR_WORKCOUNTER`，而反击主体公式又把它直接加到最终概率表达式中。因此，这类描述更准确的解释应是：

“格挡率 +X%”不是独立挡伤动作，而是直接进入反击概率链的字段。

“白石盾牌”是这一部分最合适的样例。它的说明文本写作“格挡率 +2%”，但原始数据对应的装备事件参数是 `COUNTER:20`。由于反击公式在比较随机数前先计算

$$p_{\text{counter,raw}} = 0.1F q_{\text{base}} + L + \text{CHAR_WORKCOUNTER}, \quad (12.7)$$

再乘以 100 进入 `RAND(1,10000)<per`，所以若把其它项压到 0，则单靠这件盾牌就会带来约 19.99 个百分点的真实反击率，而不是说明文本直观理解下的 2 个百分点。由此可见，盾牌上的“格挡率”文字与源码中真正进入公式的字段值并不属于同一量纲。

“迪克盾牌”可作为对照样例；而“雷甲之铠”则说明 `COUNTER` 并非盾牌专属字段，少量防具与套装同样可以直接写入反击概率链。

12.7 伤 +X / 伤-X 字段的尾段随机修正

“伤 +XXX”与“伤-XXX”最常见的误解，是把它们当作固定增伤或固定减伤。但当前源码中的真实实现更接近于“在伤害结算末段追加一个随机差值”。装备字段会把 `ITEM_OTHERDAMAGE` 写入 `CHAR_WORKOTHERDMAGE`，把 `ITEM_OTHERDEFC` 写入 `CHAR_WORKOTHERDEFC`；随后在 `BATTLE_DamageCalc()` 的末段计算

$$\Delta\text{damage} = \text{RAND}(0.3A, A) - \text{RAND}(0.3D, D), \quad (12.8)$$

其中 A 是攻方的“伤 +”字段， D 是守方的“伤-”字段。

因此，“伤 +XXX / 伤-XXX”属于伤害尾段的随机修正，而不是基础攻防层的固定加减值。

若仅看典型数值，“伤 +110”提供的大致是 33 ~ 110 的随机增伤区间；“伤-110”提供的大致是 33 ~ 110 的随机减伤区间。两边先各自取随机值，再以差值形式并入伤害尾段，所以既不能写成“固定 +110”，也不能写成“固定 -110”。

这组字段适合通过同系列装备说明。“岚黄之棍 3”的“伤 +110”应解释为“提供额外随机伤害项”；“岚黄之兜 3”“岚黄之铠 3”“焰红之铠 3”的“伤-70 / -110”则应解释为“提供额外随机减伤项”。若只写成“增伤 110”或“减伤 110”，就会误把尾段随机修正写成基础面板修正。

12.8 ITEM_MODIFYAVOID 的保守解释

关于回避类字段，还需要单列一个保守结论。当前源码确实存在

$$\text{ITEM_MODIFYAVOID} \rightarrow \text{CHAR_WORKFIXAVOID} \quad (12.9)$$

这条映射，也确实能在大量耳饰、项链、戒指以及少量防具上看到“回避率上升”的装备描述。但到目前为止，普通回避主体 `BATTLE_DuckCheck()` 并没有发现直接读取 `CHAR_WORKFIXAVOID` 的路径。

因此，`ITEM_MODIFYAVOID` 目前只能确认“字段存在且会写入 work 值”，尚不能表述为“真实普通回避率固定 +X%”。

“小耳环”“珊瑚耳饰”“五素之羽”都属于这一类道具。尤其是“五素之羽”，说明文本写作“回避率 +20%”，而物品字段实际对应 `ITEM_MODIFYAVOID=10`；即使只比较说明文本与字段实值，二者也并不一致。再进一步，由于当前尚未发现 `CHAR_WORKFIXAVOID` 进入普通回避主体，所以这类装备在正文中更合适的写法是“具有 `ITEM_MODIFYAVOID` 字段，但其对真实普通回避率的作用尚未由当前源码证实”。

换言之，`ITEM_MODIFYAVOID` 不能直接等同于“真实闪避率加成字段”；就当前源码证据而言，尚不足以将其写成普通回避率提升。

12.9 本章小结

综上，装备研究的基本方法是先确认字段写入点，再确认后续读取链，而不能直接由说明文本反推战斗作用。在这一口径下，“敏 +X”与“增加攻击顺序 X%”分别对应速度工作值与最终排序加项两条不同机制；会心的装备入口当前应按武器槽 `ITEM_CRITICAL` 理解；盾牌与套装的 `COUNTER` 字段进入的是反击概率链，而非独立格挡动作；“伤 +XXX / 伤-XXX”属于伤害尾段随机修正，而不是固定增伤或固定减伤。

同时，装备中的异常抗性、职业控制抗性与回避描述也必须分开处理。五素之羽的传统异常抗性可以确认成立，但当前源码证据不支持将其外推到盾击或天罗地网的职业控制命中；同样，`ITEM_MODIFYAVOID` 目前只能确认写入 `CHAR_WORKFIXAVOID`，尚不能直接写成真实普通回避率提升。就论文写作而言，这一章更重要的结论并不是“哪些装备更强”，而是“哪些装备描述已经被源码验证、哪些仍需保守表述”。

第十三章 属性系统的动态状态：场地、结界、加强与反转

战斗中的属性系统包含四类相互关联而又彼此独立的机制：战场级的属性场地、单位级的四属性结界、职业技能中的属性加强，以及用标志位控制的属性反转。四者进入战斗公式的位置并不相同，因此更适合被理解为一组动态状态系统，而不是单一的伤害乘法修正。本章的重点不在罗列状态名称，而在于说明这些状态如何存储、何时衰减、在何处被读取，以及为何多次命中会出现覆盖、翻转或延迟恢复等不同行为。

13.1 动态属性系统的整体框架

若把一个单位在时刻 t 的属性相关状态记为

$$Z_t = (F_t, B_t, R_t, A_t), \quad (13.1)$$

其中 F_t 表示场地状态， B_t 表示四属性结界， R_t 表示属性反转标志， A_t 表示当前生效的四属性值与抗性，那么战斗回合中的属性系统可以看成是一个离散时间状态转移系统：

$$Z_{t+1} = \mathcal{U}(Z_t, C_t, H_t), \quad (13.2)$$

其中 C_t 表示本回合动作集合， H_t 表示本回合命中结果集合， $\mathcal{U}$ 则表示回合结算、状态写入、回合衰减与重算属性的联合算子。

场地在伤害层被读取，结界在防守端减伤层被读取，属性加强若存在则应在属性重算或伤害前层进入，属性反转则在预处理重算属性时被读取。各状态的读写阶段不同，由此决定了一些状态表现为覆盖，另一些却表现为开关翻转。

13.2 场地作为战场级状态

属性场地机制对应战场结构 `BattleArray[battleindex]` 上保存的场地属性、场地强度和持续回合数。若场地属性记为 a_f ，场地强度记为 p_f ，持续回合记为 T_f ，则源码把它们分别写入

$$\text{field_att} = a_f, \quad \text{att_pow} = p_f, \quad \text{att_count} = T_f. \quad (13.3)$$

对任一单位，若其对应属性值记为 u ，则场地倍率 `FieldPow` 可整理为

$$\Phi(u, p_f) = 0.5 + \frac{up_f}{20000}. \quad (13.4)$$

由于单位属性值与场地强度都不超过 100，所以

$$0.5 \leq \Phi(u, p_f) \leq 1.0. \quad (13.5)$$

当前属性伤害在经过属性相克计算之后，还会进一步乘上攻守双方场地倍率之比：

$$damage_{\text{field}} = damage_{\text{attr}} \frac{\Phi(u_a, p_f)}{\Phi(u_d, p_f)}. \quad (13.6)$$

因此，场地不是单纯“给攻击方加伤”，而是通过攻守双方相对匹配度共同决定最终增减幅。

场地回合更新逻辑为每回合把 `att_count` 减一，当其减到非正时清空场地属性。场地是战场级、单实例、按回合整体衰减的状态。

13.3 四属性结界的覆盖写入与减伤读取

四属性结界是单位级状态，但它的存储方式与普通布尔状态不同。源码用

$$\text{MAKE2VALUE}(\text{power}, \text{turn}) \quad (13.7)$$

写入四个工作变量：`CHAR_WORKFIXEARTHAT_BOUNDARY`、`CHAR_WORKFIXWATERAT_BOUNDARY`、`CHAR_WORKFIXFIREAT_BOUNDARY`、`CHAR_WORKFIXWINDAT_BOUNDARY`。其中高 16 位存强度 p_b ，低 16 位存剩余回合 T_b 。

当前实现中，结界持续回合与强度都由技能等级分档决定。若只按源码字面逻辑整理，则持续回合近似为

$$T_b = \begin{cases} 5, & skill_level \geq 10, \\ 4, & skill_level > 9, \\ 3, & skill_level > 8, \\ 2, & skill_level > 4, \\ 1, & \text{否则}, \end{cases} \quad (13.8)$$

这一分段本身存在重叠，因此更可靠的理解方式是“以代码实际执行顺序为准”，而不是

按技能描述文本反推。强度则写成

$$p_b = \begin{cases} 100, & skill_level \geq 100, \\ 90, & skill_level > 95, \\ 80, & skill_level > 90, \\ 70, & skill_level > 85, \\ 60, & skill_level > 80, \\ 50, & skill_level > 60, \\ 40, & skill_level > 40, \\ 30, & skill_level > 20, \\ 20, & \text{否则}. \end{cases} \quad (13.9)$$

结界写入逻辑如下。若施放的是普通结界，则系统先清掉目标侧全部四种结界，再把所选属性结界写进去，即四属性结界在当前实现里是单属性覆盖关系。若施放的是“破结界”，则先按技能等级给一个破界成功率，成功后直接把该侧四种结界全部清空。

若守方存在某个属性结界，则当前物理伤害分支会按攻击方对应属性值做减伤，例如土结界对应

$$damage' = damage \left(1 - \frac{E_a}{200} \right), \quad (13.10)$$

其中 E_a 是攻方土属性值。水、火、风结界完全类比。减伤时只检查“该结界是否存在”，实际减伤量并没有把式 (13.9) 中存下来的 p_b 再乘进去。

结界机制在实战里还有几个特别容易被误判的边界场景。第一，若同一侧已经存在某种结界，再命中另一种普通结界，则旧结界不会与新结界并存，而是先被统一清空，再由新结界覆盖写入。第二，若同一种普通结界在持续回合尚未结束时再次命中，其效果更接近“按新技能结果重写当前槽位”，而不是在旧强度、旧回合数基础上继续累加。第三，若破结界命中成功，则四种结界槽会一起清空；若失败，则原结界状态整体保留。

还可以用一个具体案例看清结界与属性反转的关系。设攻击方原始四属性为

$$(E, W, F, N) = (70, 30, 0, 0), \quad (13.11)$$

分别对应土、水、火、风；再设防守方身上存在土结界。若攻击方未被反转，则土结界减伤读取的是攻方当前土属性 70，减伤比例约为 $70/200 = 35\%$ 。若攻击方先被属性反转，则当前有效属性变为

$$(E, W, F, N) = (0, 0, 70, 30). \quad (13.12)$$

此时防守方虽然仍是“土结界”，但减伤读取的已经是反转后的土属性 0，减伤比例下降为 $0/200 = 0\%$ 。这说明结界槽位本身不会被反转交换，但它所读取的“攻方对应属性值”会随着反转后的当前属性而改变。

同理，场地倍率也应理解为读取“当回合生效属性”而不是角色的某个静态建档值。若某单位长期带有属性反转标志，则其每回合预处理后参与场地倍率计算的属性向量也

会是反转后的版本。于是，属性反转不仅会改变属性相克和结界减伤，还会连带改变该单位在场地中的受益方向。

13.4 属性加强的空实现结论

当前源码中的职业共通技能“属性加强（自我强化）”对应实现仅返回 TRUE，没有实际数值逻辑。就这份代码口径而言，“自我强化”的战斗内数值效果是空实现。

由此可知，在当前代码下，多次施放属性加强不会形成层数累加，属性加强也不会与结界、场地或属性反转再叠出额外乘区，因为源码里没有可供叠加的数值入口。若后续版本或其他分支实现了具体公式，应另行分析。

13.5 属性反转的标志位与快照分离

属性反转的实现是本章最需要按时序理解的部分。技能命中后，系统先对目标的 CHAR_BATTLEFLG_REVERSE 做一次异或：

$$flg \leftarrow flg \oplus \text{REVERSE}, \quad (13.13)$$

然后立即调用 BATTLE_AttReverse()。后者并不是“无条件再交换一次”，而是先检查目标头上当前是否仍带有 REVERSE 标记；只有标记存在时，才把当前固定属性按如下映射交换：

$$(E, W, F, N) \mapsto (F, N, E, W), \quad (13.14)$$

其中 E, W, F, N 分别表示土、水、火、风，即土火互换，水风互换。

第一次命中属性反转时，标记从关变开，BATTLE_AttReverse() 随即把当前固定属性切到反转态。第二次命中同技能时，标记从开变关；此时 BATTLE_AttReverse() 进入后直接返回，不会在命中当下把当前固定属性切回原态。这表明，“反转标记被关掉”和“当前固定属性立刻复原”在源码里是两个独立事件。

使属性恢复到正常口径的，是后续的固定属性重算过程。在每回合预处理 BATTLE_PreCommandSeq() 中，系统会先重算本回合固定属性；只有头上仍带有 REVERSE 标记时，才会再次调用 BATTLE_AttReverse()。因此，若某单位已经处于反转态，又在同回合再次被属性反转命中，则会出现一个临时状态：标记已经被清掉，但当前这份固定属性快照仍然保持反转结果。等到下一次固定属性重算发生时，由于标记已关，系统不再补做交换，属性才会回到正常口径。

因此，第二次命中会关掉反转标记，但不会立即重做一次逆交换；是否在同回合内就恢复正常，取决于之后是否还有新的固定属性重算入口。若没有，这份反转后的固定属性将继续用于本回合后续判定；若有，则会在那次重算后恢复成正常属性。

由此可推出几个边际案例。第一，若单位在反转状态尚未解除时更换装备、骑宠状态改变，或本回合又被其它流程重写固定属性，则下一次重算会先按最新口径生成固定

属性，再根据标记是否仍在决定是否继续交换。反转作用的是“当回合有效属性”，而非“某次命中时刻留下的旧快照”。第二，若防守方处于反转状态而攻击方施放结界，结界写入的仍然是土、水、火、风四个固定槽位本身；被交换的是人物当前属性，而非结界槽位的语义。反转会改变结界减伤读取到的属性值，但不会把“土结界”变成“火结界”。第三，若同一目标在同回合内先被反转、后又被同技能再次命中，就可能出现“头上已无反转标记，但当前属性仍按反转态参与余下判定”的过渡现象，这要求把“标记状态”和“当前固定属性快照”分开理解。

玩家常称此效果为“属性转换”。本文沿用源码口径称其为“属性反转”，因为其本质是通过一个可开可关的标志位，在固定属性重算之后对当前属性做映射交换，而非一次性的数值替换。

13.6 多次状态命中的属性演化

把前几节合起来看，可以把多次状态命中分成三种完全不同的演化模式。

第一种是覆盖型，代表就是四属性结界。普通结界命中后会先清空已有四结界，再写入新结界，因此新状态会覆盖旧状态，而不是叠加在旧状态之上。第二种是翻转型，代表是属性反转，但这里的“翻转”严格地说只发生在标志位层；至于当前固定属性是否立刻改回去，还要看当下有没有重新进入属性重算。第三种是回合重算型，代表属性场地、结界剩余回合以及当前固定属性本身，它们会在回合推进时统一进入新的状态快照。

从目前已核对的代码可确定：结界的“多次命中”不是强度叠加而是覆盖；反转的“多次命中”不是刷新持续时间，而是对反转标志做异或；破结界的“多次命中”则表现为若干次独立清空尝试。更细一点地说，若同回合内先后发生“普通结界命中、破结界命中、再一次普通结界命中”，则最终保留的是最后一次成功写入的普通结界；若同回合内属性反转连续命中两次，则最终的标记状态会回到关闭，但当前固定属性是否已经恢复，还要看是否发生过新的固定属性重算；若同回合内既有反转又有结界，则结界写入的是槽位，反转影响的是之后读取这些槽位时所对应的当前属性值。这些都是典型的“顺序敏感”案例。

13.7 动态属性系统的状态转移模型

综合前述各节，当前属性系统最适合写成由若干算子组成的离散更新链。若把场地算子记为 $\mathcal{F}$ ，结界写入算子记为 $\mathcal{B}$ ，反转标志与属性交换的联合算子记为 $\mathcal{R}$ ，回合衰减算子记为 $\mathcal{D}$ ，则一回合内的属性状态更新可抽象为

$$Z_{t+1} = \mathcal{D} \circ \mathcal{R}^{\text{reverse}} \circ \mathcal{B}^{\text{boundary}} \circ \mathcal{F}^{\text{field}}(Z_t), \quad (13.15)$$

其中各指标函数表示本回合是否有相应状态命中。这一抽象写法揭示了属性系统的三个结构性质。第一，不同状态写入顺序不可随意交换：结界在防守端减伤层生效，反转在固

定属性重算后生效，因此“先反转再上结界”和“先上结界再反转”不一定等价。第二，反转的可逆性成立于“标志位层”和“完整重算后的新属性快照层”，连续命中两次不一定立刻把当前值切回原态。第三，属性系统是一套由战场级状态、单位级状态和标志位共同构成的动态系统，而非单一伤害乘法。

第十四章 综合讨论

14.1 经验口径与源码口径之间的张力

本文最直接的研究价值，在于把玩家社群长期积累的经验口径重新放回源码结构之中加以解释。经验判断并非没有信息含量；相反，它们往往准确捕捉到了系统中的某些局部现象。然而，一旦脱离变量入口、编译分支与离散边界，这些局部现象就容易被误写成无条件成立的普遍规则。本文前述分析显示，行动顺序、会心、反击、异常抗性以及喂药收益等多个问题都具有这一特征。

从这一角度看，源码研究并不是为了否定经验，而是为了界定经验命题成立的条件。只有在明确“哪个字段被写入”“哪个工作值被读取”“哪个分支被命中”“在哪一步发生取整与裁切”之后，经验观察才能被转化为稳定的定量判断。对于《石器时代》这样一个长期演化、口径复杂的系统而言，这种从经验命题回到源码证据的过程，本身就是研究工作的核心意义。

14.2 离散机制分析的理论含义

本文的另一个重要发现，是战斗系统与养成系统都具有显著的离散结构。战斗侧的命中、会心、反击和行动顺序并非简单连续函数，而是由单边缩放、向下取整、概率裁切和分支切换共同构成的分段系统；养成侧的转生、融合、蛋喂药与孵化同样受到单项上限、总和上限、比例回分配和多次整数截断的共同约束。二者虽然属于不同子系统，却共享“顺序敏感”“边界主导”和“局部非单调”的共同特征。

这一点解释了为何许多玩家直觉在局部区间内成立，却难以外推为全局规律。例如，“敏捷更高通常更有利”在不少局部条件下并不错，但当变量进入不同概率链并叠加离散边界后，就可能出现局部下降区间、平台区或阈值跳变。同样，“总和更高通常更强”在成长系统中也会受到压缩与取整机制的反制。由此可见，对该系统的理解不能停留在线性直觉层面，而应建立在离散状态变换与分段函数分析之上。

14.3 适用边界与后续拓展方向

尽管本文试图建立统一的源码研究框架，但其适用范围仍受到当前源码与编译路径的限制。首先，若不同服务器分支修改了宏开关、技能实现或装备读取链，则文中部分结论可能不再成立。其次，本文虽然已经对主路径进行了较系统的核对，但对极少数特殊技能、稀有交互以及异常边界条件仍采取保守口径。最后，本文的数值展示主要服务于机制解释，而非大规模实战仿真，因此在策略外推时仍需结合具体配置进一步验证。

后续可从三个方向继续展开。其一，在现有框架上补充更多分支级核验，尤其是极少数尚未完全穷尽的技能与隐藏字段。其二，将现有离散模型与更系统的数值实验结合，形成更稳定的配置图谱与阈值图谱。其三，在保留源码证据标准的前提下，把装备、阵型、AI 行为与更多实战日志进一步并入统一模型，从而使该框架不仅能够解释机制，也能够服务更高层次的构筑与培养决策。

第十五章 结论

15.1 主要研究结论

本文围绕当前 gmsv 源码，建立了一条从宠物成长、转生、融合、蛋喂药到战斗速度、物理概率、法术命中、属性状态与最终伤害的统一分析链。就宠物系统而言，本文区分了基础成长、波动后生效成长、蛋基础成长、孵化后成长与出生后随机状态，并将转生、融合、孵化与喂药还原为带多次取整、上限裁切与比例回分配的离散映射。就战斗系统而言，本文系统整理了行动顺序的速度分支、骑宠参照敏捷、普通回避 / 命中 / 会心 / 反击公式、法术命中与伤害分层、场地与结界、属性反转以及若干职业状态变量的真实作用位置。

在此基础上，可以得到若干能够直接修正玩家经验口径的结论。第一，行动顺序不能仅按“动作名称”理解，而必须先区分默认分支、普通物品分支、精灵 / 普通法术分支与专用加速分支；其中普通物品与精灵都只读取人物本体 WORKQUICK，而不读取骑宠混合后的参照敏捷。第二，装备研究不能按说明文字直接分型，而必须回到源码字段入口；“增加攻击顺序”不等于“敏捷上升”，“格挡率”不等于独立挡伤动作，“伤 + / 伤-”也不等于固定增伤 / 减伤。第三，五素之羽这类装备的传统异常抗性可以确认进入传统异常状态链，但不会影响盾击或天罗地网这类职业控制命中；ITEM_MODIFYAVOID 当前也不能直接写成真实普通回避率加成。第四，命中、会心与反击对敏捷都不是全局单调函数，取整与边界裁切会进一步放大局部非单调与平台区现象。第五，蛋喂药与成长压缩问题天然属于离散优化问题，其最优策略取决于取整顺序与压缩边界，而不服从“总和越高越优”之类的线性直觉。

15.2 对经验口径的修正

本文修正了若干常见但并不严格成立的经验说法。其一，“敏捷越高越容易命中或会心”并不成立；在当前源码口径下，命中与会心都存在局部下降区间。其二，“骑宠一定能让人物所有动作更快”并不成立；只有走默认速度分支的动作才真正读取骑宠参照敏捷，普通物品、精灵与普通法术都不读取该值。其三，“格挡率 +X%”与“会心 +X”不能按说明文本一比一外推成真实概率增量；装备字段进入公式后的量纲与说明文字往往并不相同。其四，“回避率上升”类装备当前也不能直接当作普通回避率既定加成，因为

尚未发现 CHAR_WORKFIXAVOID 进入普通回避主体的路径。其五，“全部喂 5 级药一定最优”同样不成立，因为多次取整与总和压缩会改变最优方案的边界。

这些修正并不是要否定经验观察本身，而是要指出经验结论成立的条件、变量口径与适用边界。源码分析的意义，正在于把“局部有效的玩家直觉”还原成“何时成立、何时失效的定量命题”。

15.3 研究局限与后续工作

本文仍有若干明确的局限。首先，全文口径以当前持有的 gmsv 源码与当前编译路径为准；不同分支、不同宏组合或后续版本可能改变部分实现细节。其次，本文虽然已经建立了从养成到战斗的完整分析链，但在数值实验层面仍以结构分析、局部公式推导与典型图形为主，尚未将所有章节扩展为大规模仿真。再次，若干机制虽然已经核清主路径，但重复命中、跨状态覆盖、异常边界条件与特殊职业技能的全部分支仍未穷尽验证。

因此，后续工作主要有三方面。第一，继续对少数仍保守处理的路径做分支级校核，尤其是装备隐藏属性、特殊技能与跨状态交互。第二，在现有离散模型基础上补充更系统的数值实验，把章节中的局部结论扩展为更完整的配置图谱。第三，把装备、行为与更多实战日志进一步并入统一模型，使“源码可证的定量框架”能够更直接服务于实际构筑与培养决策。

总的来看，本文表明：面对一个充满经验说法的老游戏系统，只要回到源码、变量入口、离散取整与状态转移本身，仍然能够建立一套自洽、可验证且可复用的理论框架。对《石器时代》这样兼具战斗深度与养成复杂度的系统而言，这种“先统一口径，再讨论优化”的研究路径是必要的。

附录 A 符号表

本附录给出完整符号表。第 2 章表 ?? 与表 ?? 只列出核心记号，详见本表。

符号 / 变量	对应对象	说明
V, S, T, D	宠物四项成长率	分别对应生命、攻击、防御、敏捷方向的成长来源
g	基础成长档	正文主记号，一般写作 (V, S, T, D)
(V, S, T, D)	玩家口径别名	玩家或工具口径常把基础成长档直接写成 (V, S, T, D) ；正文中把它视为 g 的别名
$VSTD$	成长总和	定义为 $V + S + T + D$
$g^{(new)}$	转生后成长档	转生公式输出的确定性新成长率
b	蛋基础成长	融合完成后、喂药前或喂药后的蛋面值
h	孵化后成长	按孵化公式折算后的成长，不含出生后 10 点随机
HP, ATK, DEF, SPD	宠物面板四围	界面中显示的生命、攻击、防御、敏捷数值
$INITNUM$	一级初始化尺度	宠物出生时 raw 点数的乘数基数
$g_i^{(s)}$	生效成长率	经 ± 2 波动后真正存入成长档的成长率
e_i	出生随机加点	一级出生时额外随机分配的 10 点
$p_i^{(\ell)}$	升级随机加点	第 ℓ 次升级时随机分配的 10 点
$fRand$	升级倍率	由 Rank 决定取值区间的升级随机倍率
A	转生能力值	工具内部旧称 <code>ans</code> ，文中统一称“转生能力值”

符号 / 变量	对应对象	说明
$A_{\max}$	转生能力上限	首转默认为 150，二转默认为 200，可人工覆盖
att_{dex}	攻击方固定敏	玩家 PVP 概率分析中的攻方固定敏记号
def_{dex}	防守方固定敏	玩家 PVP 概率分析中的守方固定敏记号
Q	战斗速度敏	一般对应 WORKQUICK，主要进入行动顺序
DEX_{fix}	固定敏	一般对应 WORKFIXDEX，主要进入回避、会心、反击
L	幸运	需区分原始幸运 CHAR_LUCK 与固定幸运 WORKFIXLUCK
C	武器会心值	普通物理会心公式中的武器附加参数
F	反击武器系数	玩家反击率中的武器对武器系数
α	回避参数	物理回避公式中的主参数，常见取值为 0.02 或 0.027
β	会心参数	物理会心公式中的主参数，主分支常见取值为 0.09
γ	反击参数	物理反击公式中的主参数，主分支常见取值为 0.08
p_{duck}	普通回避率	不含独立前判的普通闪避概率
p_{hit}	普通命中率	通常定义为 $100\% - p_{\text{duck}}$
p_{crit}	会心率	当前为普通物理会心判定
p_{counter}	反击率	在满足反击前提后的成功概率
R, S, G	抗性相关量	常分别表示对应抗性、抗装、精灵抗性
P	熟练度	法术或职业技能对应属性熟练
F_t	场地状态	包含场地属性、场地强度与回合数
B_t	结界状态	四属性结界的单位级状态
E_t	属性附体状态	火附体、冰附体、雷附体等持续状态
R_t	反转状态	属性反转开关标志
Z_t	动态属性状态向量	统一表示场地、结界、属性附体、反转和当前属性

符号 / 变量	对应对象	说明
U	目标函数	表示战斗或培养上的效用函数
U_0	代理目标	通常为孵化后成长空间上的线性或词典序目标
Ψ	复合映射	表示从培养输入到战斗输出的整体映射
WORKFIXDEX	固定敏工作值	物理概率公式常直接读取
WORKQUICK	速度工作值	行动顺序公式常直接读取
CHAR_BATTLEF LG_REVERSE	属性反转标志	通过异或切换，触发属性互换
CHAR_BATTLEF LG_AIBAD	本回合忠诚失控标志 (nono)	由 BATTLE_PetLoyalCheck() 按回合置位，表示宠物未按原指令行动
CHAR_WORKBAT TLECOM3	动作附加参数槽	高低位会被不同技能复用
MAKE2VALUE (power, turn)	高低位打包状态	结界等状态用高位存强度、低位存回合

附录 B 公式总表

本附录汇总正文中的关键公式与约束条件，便于从“想查哪条公式”反向定位到相应章节。表中的“编号”使用正文中的公式标签。

编号	所在章节	含义
eq:pet-display -map	第 3 章	宠物原始累计点到显示 HP/ATK/DEF/SPD 的映射
eq:lv1-raw	第 3 章	一级初始化 raw 点数公式
eq: levelup-delta	第 3 章	升级时每一维 raw 点增长量
eq: trans-ans-raw	第 4 章	转生能力值原始公式
eq: trans-ans-cap	第 4 章	转生能力上限裁切
eq:trans-new-a lloc	第 4 章	转生能力值按权重分配到四维成长
eq:fusion-lev el-discount	第 4 章	低于 80 级融合素材的逐项折扣
eq:sub-avg	第 4 章	副宠逐项平均的取整公式
eq:egg-base	第 4 章	蛋基础成长生成公式
eq:feed-fold	第 4 章	蛋药原始值向工作值的折算公式
eq:work-cap	第 4 章	50 总和压缩
eq:hatch-pre	第 4 章	孵化前的初步成长生成公式
eq:hatch-final	第 4 章	最终 150 总和压缩
eq:turn-normal	第 7 章	默认乱敏顺序值公式
eq:turn-item	第 7 章	道具使用的顺序值公式
eq:turn-quick	第 7 章	迅速攻击类顺序值公式

编号	所在章节	含义
eq:turn-ref-mounted	第 7 章	骑宠状态下默认档参照敏捷公式
eq:dodge-base	第 8 章	玩家 PVP 简化回避基础体
eq:dodge-final	第 8 章	玩家打玩家时的源码离散回避率公式
eq:hit-final	第 8 章	玩家打玩家时的源码离散命中率公式
eq:crit-final	第 8 章	玩家打玩家时的源码离散会心率公式
eq:counter-final	第 8 章	玩家打玩家时的源码离散反击率公式
eq:hit-derivative	第 8 章	命中率对攻方敏捷的一阶导表达式
eq:crit-derivative	第 8 章	会心率在 $att_{dex} < def_{dex}$ 区间的一阶导表达式
eq:counter-derivative	第 8 章	反击率在 $att_{dex} < def_{dex}$ 区间的一阶导表达式
eq:magic-miss-player	第 9 章	普通属性法术打玩家的未命中基数
eq:magic-hit-player	第 9 章	普通属性法术打玩家的命中率
eq:prof-miss-player	第 9 章	职业法术打玩家的第一段未命中基数
eq:prof-hit-stage1	第 9 章	职业法术第一段命中率
eq:prof-hit-stage2	第 9 章	职业法术二段判定的组合命中率
eq:magic-damage-layer	第 9 章	职业法术命中后的伤害层
eq:fieldpow	第 10 章	场地倍率 FieldPow 公式
eq:field-damage	第 10 章	场地对属性伤害的缩放位置
eq:boundary-power	第 10 章	结界强度的分段规则
eq:ice-enclose	第 10 章	冰附体对固定敏的作用式
eq:reverse-map	第 10 章	属性反转的四属性互换映射

编号	所在章节	含义
eq:ride-atk-ranged	第 11 章	骑宠远程攻击混合公式
eq:ride-atk-melee	第 11 章	骑宠近战攻击混合公式
eq:def-no-ride	第 11 章	未骑宠时进入物伤公式的防御量
eq:def-ride	第 11 章	骑宠时进入物伤公式的防御量
eq:ride-quick-ranged	第 11 章	骑宠远程攻击侧速度混合
eq:ride-quick-melee	第 11 章	骑宠近战攻击侧速度混合
eq:ride-quick-def	第 11 章	骑宠防守侧速度混合
eq:damage-stage1	第 11 章	物理伤害第一段
eq:damage-stage2	第 11 章	物理伤害第二段
eq:damage-stage3	第 11 章	物理伤害第三段
eq:crit-damage	第 11 章	会心附加伤害公式
eq:growth-main-path	第 12 章	成长维度到战斗属性的主路径映射
eq:linear-hatch-target	第 12 章	孵化后成长空间上的线性代理目标
eq:global-opt	第 13 章	全局培养优化问题的统一形式
eq:feed-constraint	第 13 章	40 药约束
eq:hatch-opt	第 13 章	蛋喂药的中间优化问题

附录 C 源码索引

本附录记录正文中最关键机制所对应的源码文件、函数和已核对的关键区段，便于后续继续追源或校核。由于当前论文同时参考了游戏源码与本地工具实现，表中也标出少量工具侧对齐实现。

机制	文件 / 函数	关键位置	说明
普通宠 Rank 表	enemy.c / ENEMY_getRank	工具对齐	0 转宠 Rank 边界在工具中 由 backend/simulator.py 对齐实现
转生宠 Rank 表	enemy.c / GetNewPet 内部	工具对齐	转生宠 Rank 边界在工具中 由 backend/simulator.py 对齐实现
升级倍率表	char_data.c / RankRandTbl	工具对齐	各 Rank 的 $fRand$ 区间
成长模拟	tools/pet_revers e/backend/simula tor.py	全文件	对齐普通宠与转生宠的成长 随机流程
转生计算	tools/pet_revers e/backend/simula tor.py / calc_tra nsmigration	全文件	对齐转生能力值与分配规则
融合与孵化	tools/pet_revers e/backend/fusion .py、evolution.py	全文件	对齐融合成蛋与蛋喂药整数 规则
宠物忠诚失控 (nono)	src/battle/battle. c / BATTLE_PetLoya lCheck	6802–6935	读取 CHAR_WORKFIXAI，按区 间概率改写宠物动作，并置 CHAR_BATTLEFLG_AIBAD

机制	文件 / 函数	关键位置	说明
随机宠技改写	src/battle/battle.c / BATTLE_PetRandomSkill	6708–6788	在“随机动作”模式下改放另一项战斗宠技；若新技能不重写工作值，可保留原技能已写入的攻防速修正
属性反转开关	src/battle/battle_magic.c	2345–2352	反转标志异或切换，并立即调用 BATTLE_AttReverse()
属性反转换映	src/battle/battle.c / BATTLE_AttReverse	2880–2900	土火互换、水风互换
回合预处理中的反转	src/battle/battle.c / BATTLE_PreCommandSeq	2956–2958 附近	若反转标志存在，则重算固定属性后再次交换
结界回合衰减	src/battle/battle.c	8814–8865	低 16 位减一，减到负值时清空
场地伤害倍率	src/battle/battle_event.c / BATTLE_AttrAdjustPath	1074–1130	通过 BATTLE_FieldAttAdjust() 生成攻守双方 FieldPow
结界减伤	src/battle/battle_event.c	1243–1255	只检查结界是否存在，减伤主要看攻方对应属性值
四属性结界写入	src/battle/battle_event.c / BATTLE_COM_S_BOUNDARY	9313–9405	普通结界先清四结界再写入；破结界可清空全部
火/冰/雷附体包装	src/battle/profession_skill.c	881–899	三种封印技能的包装入口
四属性结界包装	src/battle/profession_skill.c	1160–1163	结界技能入口
自我强化空实现	src/battle/profession_skill.c / PROFESSION_strengthen	1288–1291	当前源码仅返回 TRUE，未见数值逻辑